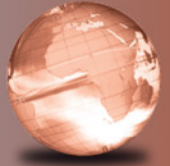


GLOBAL
EDITION

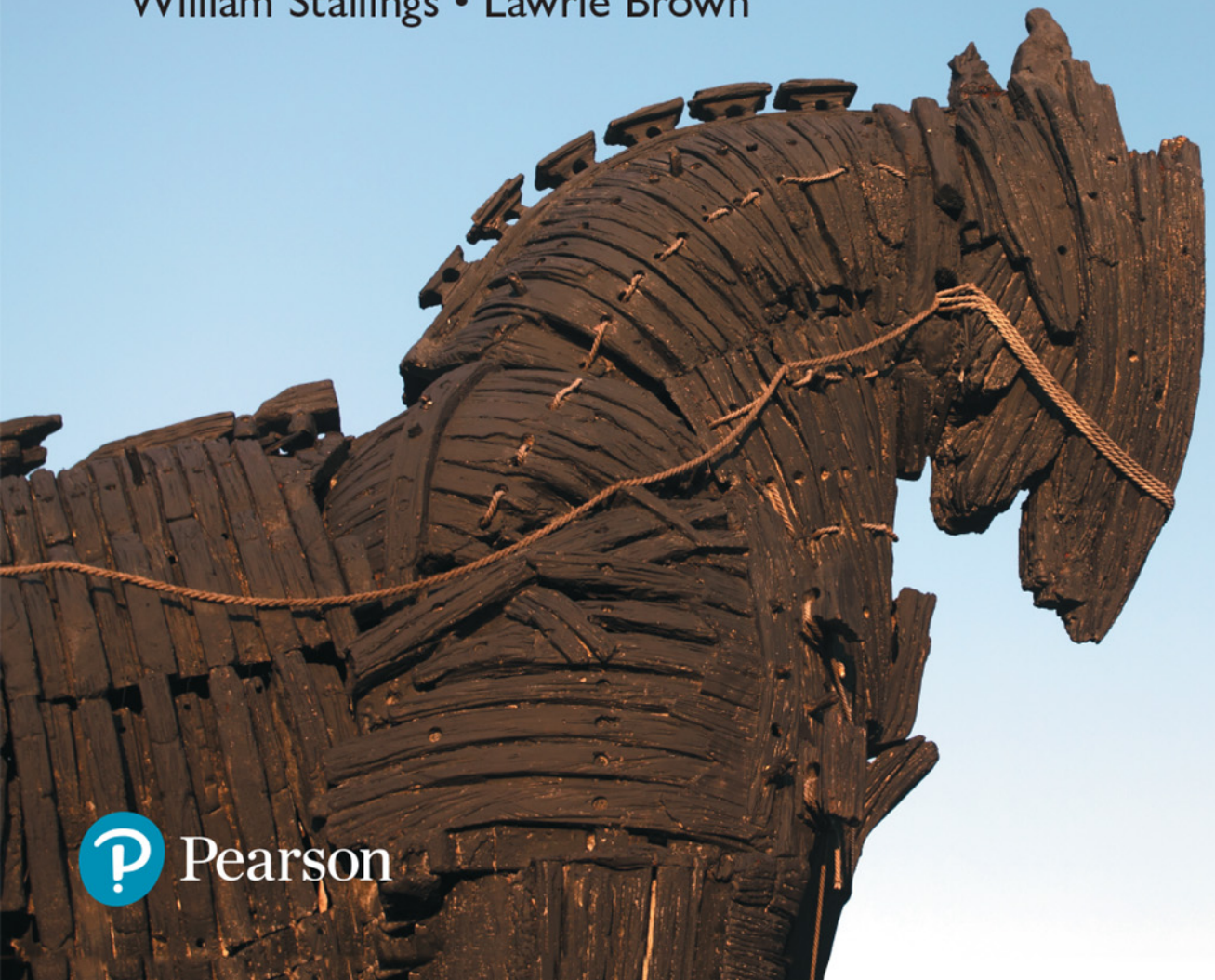


Computer Security

Principles and Practice

FOURTH EDITION

William Stallings • Lawrie Brown



Pearson

Digital Resources for Students

Your new textbook provides 12-month access to digital resources that may include VideoNotes (step-by-step video tutorials on programming concepts), source code, web chapters, quizzes, and more. Refer to the preface in the textbook for a detailed list of resources.

Follow the instructions below to register for the Companion Website for William Stallings/Lawrie Brown's *Computer Security: Principles and Practice*, Fourth Edition, Global Edition.

1. Go to www.pearsonglobaleditions.com/stallings.
2. Enter the title of your textbook or browse by author name.
3. Click Companion Website.
4. Click Register and follow the on-screen instructions to create a login name and password.

**Use a coin to scratch off the coating and reveal your access code.
Do not use a sharp knife or other sharp object as it may damage the code.**

Use the login name and password you created during registration to start using the online resources that accompany your textbook.

IMPORTANT:

This access code can only be used once. This subscription is valid for 12 months upon activation and is not transferrable. If the access code has already been revealed it may no longer be valid.

For technical support go to <https://support.pearson.com/getsupport/>

COMPUTER SECURITY

PRINCIPLES AND PRACTICE

Fourth Edition

Global Edition

William Stallings

Lawrie Brown

UNSW Canberra at the Australian Defence Force Academy



330 Hudson Street, New York, NY 10013

**Director, Portfolio Management: Engineering,
Computer Science & Global Editions:**
Julian Partridge
Specialist, Higher Ed Portfolio Management:
Tracy Johnson (Dunkelberger)
Acquisitions Editor, Global Edition: Sourabh
Maheshwari
Portfolio Management Assistant: Meghan Jacoby
Managing Content Producer: Scott Disanno
Content Producer: Robert Engelhardt
Project Editor, Global Edition: K.K. Neelakantan
Web Developer: Steve Wright
Manager, Media Production, Global Edition: Vikram
Kumar

Rights and Permissions Manager: Ben Ferrini
**Manufacturing Buyer, Higher Ed, Lake Side
Communications Inc (LSC):** Maura Zaldivar-Garcia
Senior Manufacturing Controller, Global Edition:
Angela Hawksbee
Inventory Manager: Ann Lam
Product Marketing Manager: Yvonne Vannatta
Field Marketing Manager: Demetrius Hall
Marketing Assistant: Jon Bryant
Cover Designer: Lumina Datamatics, Inc.
Cover Photo: Alex Kosev / Shutterstock
Full-Service Project Management: Kirthika Raj,
SPi Global

Credits and acknowledgments borrowed from other sources and reproduced, with permission, in this textbook appear on page 777.

Many of the designations by manufacturers and seller to distinguish their products are claimed as trademarks. Where those designations appear in this book, and the publisher was aware of a trademark claim, the designations have been printed in initial caps or all caps.

Pearson Education Limited
KAO Two
KAO Park
Harlow
CM17 9NA
United Kingdom

and Associated Companies throughout the world

Visit us on the World Wide Web at:
www.pearsonglobaleditions.com

© Pearson Education Limited 2018

The rights of William Stallings and Lawrie Brown to be identified as the authors of this work have been asserted by them in accordance with the Copyright, Designs and Patents Act 1988.

Authorized adaptation from the United States edition, entitled Computer Security: Principles and Practice, 4th Edition, ISBN 978-0-13-479410-5 by William Stallings and Lawrie Brown published by Pearson Education © 2018.

All rights reserved. No part of this publication may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, electronic, mechanical, photocopying, recording or otherwise, without either the prior written permission of the publisher or a license permitting restricted copying in the United Kingdom issued by the Copyright Licensing Agency Ltd, Saffron House, 6–10 Kirby Street, London EC1N 8TS.

All trademarks used herein are the property of their respective owners. The use of any trademark in this text does not vest in the author or publisher any trademark ownership rights in such trademarks, nor does the use of such trademarks imply any affiliation with or endorsement of this book by such owners.

British Library Cataloguing-in-Publication Data
A catalogue record for this book is available from the British Library

10 9 8 7 6 5 4 3 2 1

ISBN 10: 1-292-22061-9
ISBN 13: 978-1-292-22061-1

Typeset by SPi Global
Printed and bound in Malaysia

CONTENTS

Preface 12

Notation 21

About the Authors 22

Chapter 1 Overview 23

- 1.1 Computer Security Concepts 24
- 1.2 Threats, Attacks, and Assets 31
- 1.3 Security Functional Requirements 37
- 1.4 Fundamental Security Design Principles 39
- 1.5 Attack Surfaces and Attack Trees 43
- 1.6 Computer Security Strategy 46
- 1.7 Standards 48
- 1.8 Key Terms, Review Questions, and Problems 49

PART ONE COMPUTER SECURITY TECHNOLOGY AND PRINCIPLES 52

Chapter 2 Cryptographic Tools 52

- 2.1 Confidentiality with Symmetric Encryption 53
- 2.2 Message Authentication and Hash Functions 59
- 2.3 Public-Key Encryption 67
- 2.4 Digital Signatures and Key Management 72
- 2.5 Random and Pseudorandom Numbers 77
- 2.6 Practical Application: Encryption of Stored Data 79
- 2.7 Key Terms, Review Questions, and Problems 80

Chapter 3 User Authentication 85

- 3.1 Digital User Authentication Principles 86
- 3.2 Password-Based Authentication 92
- 3.3 Token-Based Authentication 104
- 3.4 Biometric Authentication 109
- 3.5 Remote User Authentication 114
- 3.6 Security Issues for User Authentication 117
- 3.7 Practical Application: An Iris Biometric System 119
- 3.8 Case Study: Security Problems for ATM Systems 121
- 3.9 Key Terms, Review Questions, and Problems 124

Chapter 4 Access Control 127

- 4.1 Access Control Principles 128
- 4.2 Subjects, Objects, and Access Rights 131
- 4.3 Discretionary Access Control 132
- 4.4 Example: UNIX File Access Control 139
- 4.5 Role-Based Access Control 142
- 4.6 Attribute-Based Access Control 148

6 CONTENTS

4.7	Identity, Credential, and Access Management	154
4.8	Trust Frameworks	158
4.9	Case Study: RBAC System for a Bank	162
4.10	Key Terms, Review Questions, and Problems	164

PREFACE

WHAT'S NEW IN THE FOURTH EDITION

Since the third edition of this book was published, the field has seen continued innovations and improvements. In this new edition, we try to capture these changes while maintaining a broad and comprehensive coverage of the entire field. To begin the process of revision, the third edition of this book was extensively reviewed by a number of professors who teach the subject and by professionals working in the field. The result is that in many places the narrative has been clarified and tightened, and illustrations have been improved.

Beyond these refinements to improve pedagogy and user-friendliness, there have been major substantive changes throughout the book. The most noteworthy changes are as follows:

- **Data center security:** Chapter 5 includes a new discussion of data center security, including the TIA-492 specification of reliability tiers.
- **Malware:** The material on malware in Chapter 6 has been revised to include additional material on macro viruses and their structure, as they are now the most common form of virus malware.
- **Virtualization security:** The material on virtualization security in Chapter 12 has been extended, given the rising use of such systems by organizations and in cloud computing environments. A discussion of virtual firewalls, which may be used to help secure these environments, has also been added.
- **Cloud security:** Chapter 13 includes a new discussion of cloud security. The discussion includes an introduction to cloud computing, key cloud security concepts, an analysis of approaches to cloud security, and an open-source example.
- **IoT security:** Chapter 13 includes a new discussion of security for the Internet of Things (IoT). The discussion includes an introduction to IoT, an overview of IoT security issues, and an open-source example.
- **SEIM:** The discussion of Security Information and Event Management (SIEM) systems in Chapter 18 has been updated.
- **Privacy:** The section on privacy issues and its management in Chapter 19 has been extended with additional discussion of moral and legal approaches, and the privacy issues related to big data.
- **Authenticated encryption:** Authenticated encryption has become an increasingly widespread cryptographic tool in a variety of applications and protocols. Chapter 21 includes a new discussion of authenticated description and describes an important authenticated encryption algorithm known as offset codebook (OCB) mode.

BACKGROUND

Interest in education in computer security and related topics has been growing at a dramatic rate in recent years. This interest has been spurred by a number of factors, two of which stand out:

1. As information systems, databases, and Internet-based distributed systems and communication have become pervasive in the commercial world, coupled with the increased intensity and sophistication of security-related attacks, organizations now recognize the need for a comprehensive security strategy. This strategy encompasses the use of specialized hardware and software and trained personnel to meet that need.
2. Computer security education, often termed *information security education* or *information assurance education*, has emerged as a national goal in the United States and other countries, with national defense and homeland security implications. The NSA/DHS National Center of Academic Excellence in Information Assurance/Cyber Defense is spearheading a government role in the development of standards for computer security education.

Accordingly, the number of courses in universities, community colleges, and other institutions in computer security and related areas is growing.

OBJECTIVES

The objective of this book is to provide an up-to-date survey of developments in computer security. Central problems that confront security designers and security administrators include defining the threats to computer and network systems, evaluating the relative risks of these threats, and developing cost-effective and user friendly countermeasures.

The following basic themes unify the discussion:

- **Principles:** Although the scope of this book is broad, there are a number of basic principles that appear repeatedly as themes and that unify this field. Examples are issues relating to authentication and access control. The book highlights these principles and examines their application in specific areas of computer security.
- **Design approaches:** The book examines alternative approaches to meeting specific computer security requirements.
- **Standards:** Standards have come to assume an increasingly important, indeed dominant, role in this field. An understanding of the current status and future direction of technology requires a comprehensive discussion of the related standards.
- **Real-world examples:** A number of chapters include a section that shows the practical application of that chapter's principles in a real-world environment.

SUPPORT OF ACM/IEEE COMPUTER SCIENCE CURRICULA 2013

This book is intended for both an academic and a professional audience. As a textbook, it is intended as a one- or two-semester undergraduate course for computer science, computer engineering, and electrical engineering majors. This edition is designed to support

Table P.1 Coverage of CS2013 Information Assurance and Security (IAS) Knowledge Area

IAS Knowledge Units	Topics	Textbook Coverage
Foundational Concepts in Security (Tier 1)	• CIA (Confidentiality, Integrity, and Availability) • Risk, threats, vulnerabilities, and attack vectors • Authentication and authorization, access control (mandatory vs. discretionary) • Trust and trustworthiness • Ethics (responsible disclosure)	1—Overview 3—User Authentication 4—Access Control 19—Legal and Ethical Aspects
Principles of Secure Design (Tier 1)	• Least privilege and isolation • Fail-safe defaults • Open design • End-to-end security • Defense in depth • Security by design • Tensions between security and other design goals	1—Overview
Principles of Secure Design (Tier 2)	• Complete mediation • Use of vetted security components • Economy of mechanism (reducing trusted computing base, minimize attack surface) • Usable security • Security composability • Prevention, detection, and deterrence	1—Overview
Defensive Programming (Tier 1)	• Input validation and data sanitization • Choice of programming language and type-safe languages • Examples of input validation and data sanitization errors (buffer overflows, integer errors, SQL injection, and XSS vulnerability) • Race conditions • Correct handling of exceptions and unexpected behaviors	11—Software Security
Defensive Programming (Tier 2)	• Correct usage of third-party components • Effectively deploying security updates	11—Software Security 12—OS Security
Threats and Attacks (Tier 2)	• Attacker goals, capabilities, and motivations • Malware • Denial of service and distributed denial of service • Social engineering	6—Malicious Software 7—Denial-of-Service Attacks
Network Security (Tier 2)	• Network-specific threats and attack types • Use of cryptography for data and network security • Architectures for secure networks • Defense mechanisms and countermeasures • Security for wireless, cellular networks	8—Intrusion Detection 9—Firewalls and Intrusion Prevention Systems Part 5—Network Security
Cryptography (Tier 2)	• Basic cryptography terminology • Cipher types • Overview of mathematical preliminaries • Public key infrastructure	2—Cryptographic Tools Part 4—Cryptographic Algorithms

the recommendations of the ACM/IEEE Computer Science Curricula 2013 (CS2013). The CS2013 curriculum recommendation includes, for the first time, Information Assurance and Security (IAS) as one of the Knowledge Areas in the Computer Science Body of Knowledge. CS2013 divides all course work into three categories: Core-Tier 1 (all topics should be included in the curriculum), Core-Tier 2 (all or almost all topics should be included), and Elective (desirable to provide breadth and depth). In the IAS area, CS2013 includes three Tier 1 topics, five Tier 2 topics, and numerous Elective topics, each of which has a number of subtopics. This text covers all of the Tier 1 and Tier 2 topics and subtopics listed by CS2013, as well as many of the elective topics. Table P.1 shows the support for the ISA Knowledge Area provided in this textbook.

COVERAGE OF CISSP SUBJECT AREAS

This book provides coverage of all the subject areas specified for CISSP (Certified Information Systems Security Professional) certification. The CISSP designation from the International Information Systems Security Certification Consortium (ISC)² is often referred to as the “gold standard” when it comes to information security certification. It is the only universally recognized certification in the security industry. Many organizations, including the U.S. Department of Defense and many financial institutions, now require that cyber security personnel have the CISSP certification. In 2004, CISSP became the first IT program to earn accreditation under the international standard ISO/IEC 17024 (*General Requirements for Bodies Operating Certification of Persons*).

The CISSP examination is based on the Common Body of Knowledge (CBK), a compendium of information security best practices developed and maintained by (ISC)², a nonprofit organization. The CBK is made up of 8 domains that comprise the body of knowledge that is required for CISSP certification.

The 8 domains are as follows, with an indication of where the topics are covered in this textbook:

- **Security and risk management:** Confidentiality, integrity, and availability concepts; security governance principles; risk management; compliance; legal and regulatory issues; professional ethics; and security policies, standards, procedures, and guidelines. (*Chapter 14*)
- **Asset security:** Information and asset classification; ownership (e.g. data owners, system owners); privacy protection; appropriate retention; data security controls; and handling requirements (e.g., markings, labels, storage). (*Chapters 5, 15, 16, 19*)
- **Security engineering:** Engineering processes using secure design principles; security models; security evaluation models; security capabilities of information systems; security architectures, designs, and solution elements vulnerabilities; web-based systems vulnerabilities; mobile systems vulnerabilities; embedded devices and cyber-physical systems vulnerabilities; cryptography; and site and facility design secure principles; physical security. (*Chapters 1, 2, 13, 15, 16*)
- **Communication and network security:** Secure network architecture design (e.g., IP and non-IP protocols, segmentation); secure network components; secure communication channels; and network attacks. (*Part Five*)

- **Identity and access management:** Physical and logical assets control; identification and authentication of people and devices; identity as a service (e.g. cloud identity); third-party identity services (e.g., on-premise); access control attacks; and identity and access provisioning lifecycle (e.g., provisioning review). (*Chapters 3, 4, 8, 9*)
- **Security assessment and testing:** Assessment and test strategies; security process data (e.g., management and operational controls); security control testing; test outputs (e.g., automated, manual); and security architectures vulnerabilities. (*Chapters 14, 15, 18*)
- **Security operations:** Investigations support and requirements; logging and monitoring activities; provisioning of resources; foundational security operations concepts; resource protection techniques; incident management; preventative measures; patch and vulnerability management; change management processes; recovery strategies; disaster recovery processes and plans; business continuity planning and exercises; physical security; and personnel safety concerns. (*Chapters 11, 12, 15, 16, 17*)
- **Software development security:** Security in the software development lifecycle; development environment security controls; software security effectiveness; and acquired software security impact. (*Part Two*)

SUPPORT FOR NSA/DHS CERTIFICATION

The U.S. National Security Agency (NSA) and the U.S. Department of Homeland Security (DHS) jointly sponsor the National Centers of Academic Excellence in Information Assurance/Cyber Defense (IA/CD). The goal of these programs is to reduce vulnerability in our national information infrastructure by promoting higher education and research in IA and producing a growing number of professionals with IA expertise in various disciplines. To achieve that purpose, NSA/DHS have defined a set of Knowledge Units for 2- and 4-year institutions that must be supported in the curriculum to gain a designation as a NSA/DHS National Center of Academic Excellence in IA/CD. Each Knowledge Unit is composed of a minimum list of required topics to be covered and one or more outcomes or learning objectives. Designation is based on meeting a certain threshold number of core and optional Knowledge Units.

In the area of computer security, the 2014 Knowledge Units document lists the following core Knowledge Units:

- **Cyber Defense:** Includes access control, cryptography, firewalls, intrusion detection systems, malicious activity detection and countermeasures, trust relationships, and defense in depth.
- **Cyber Threats:** Includes types of attacks, legal issues, attack surfaces, attack trees, insider problems, and threat information sources.
- **Fundamental Security Design Principles:** A list of 12 principles, all of which are covered in Section 1.4 of this text.
- **Information Assurance Fundamentals:** Includes threats and vulnerabilities, intrusion detection and prevention systems, cryptography, access control models, identification/authentication, and audit.

- **Introduction to Cryptography:** Includes symmetric cryptography, public-key cryptography, hash functions, and digital signatures.
- **Databases:** Includes an overview of databases, database access controls, and security issues of inference.

This book provides extensive coverage in all of these areas. In addition, the book partially covers a number of the optional Knowledge Units.

PLAN OF THE TEXT

The book is divided into five parts (see Chapter 0):

- Computer Security Technology and Principles
- Software and System Security
- Management Issues
- Cryptographic Algorithms
- Network Security

The text is also accompanied by a number of online chapters and appendices that provide more detail on selected topics.

The text includes an extensive glossary, a list of frequently used acronyms, and a bibliography. Each chapter includes homework problems, review questions, a list of key words, and suggestions for further reading.

INSTRUCTOR SUPPORT MATERIALS

The major goal of this text is to make it as effective a teaching tool for this exciting and fast-moving subject as possible. This goal is reflected both in the structure of the book and in the supporting material. The text is accompanied by the following supplementary material to aid the instructor:

- **Projects manual:** Project resources including documents and portable software, plus suggested project assignments for all of the project categories listed in the following section.
- **Solutions manual:** Solutions to end-of-chapter Review Questions and Problems.
- **PowerPoint slides:** A set of slides covering all chapters, suitable for use in lecturing.
- **PDF files:** Reproductions of all figures and tables from the book.
- **Test bank:** A chapter-by-chapter set of questions.
- **Sample syllabuses:** The text contains more material than can be conveniently covered in one semester. Accordingly, instructors are provided with several sample syllabuses that guide the use of the text within limited time. These samples are based on real-world experience by professors with the first edition.

All of these support materials are available at the Instructor Resource Center (IRC) for this textbook, which can be reached through the publisher's Website www.pearsonglobaleditions.com/stallings. To gain access to the IRC, please contact your local Pearson sales representative.

The **Companion Website** includes the following:

- Links to Web sites for other courses being taught using this book.
- Sign-up information for an Internet mailing list for instructors using this book to exchange information, suggestions, and questions with each other and with the author.

STUDENT RESOURCES



For this new edition, a tremendous amount of original supporting material for students has been made available online, at two Web locations. The **Companion Website**, includes a list of relevant links organized by chapter and an errata sheet for the book.

Purchasing this textbook now grants the reader 12 months of access to the **Premium Content Site**, which includes the following materials:

- **Online chapters:** To limit the size and cost of the book, three chapters of the book are provided in PDF format. The chapters are listed in this book's table of contents.
- **Online appendices:** There are numerous interesting topics that support material found in the text but whose inclusion is not warranted in the printed text. A total of eleven online appendices cover these topics for the interested student. The appendices are listed in this book's table of contents.
- **Homework problems and solutions:** To aid the student in understanding the material, a separate set of homework problems with solutions is available. These enable the students to test their understanding of the text.

To access the Premium Content site, click on the link at www.pearsonglobaleditions.com/stallings and enter the student access code found on the inside front cover.

PROJECTS AND OTHER STUDENT EXERCISES

For many instructors, an important component of a computer security course is a project or set of projects by which the student gets hands-on experience to reinforce concepts from the text. This book provides an unparalleled degree of support for including a projects component in the course. The instructor's support materials available through Pearson not only include guidance on how to assign and structure the projects but also include a set of user manuals for various project types plus specific assignments, all written especially for this book. Instructors can assign work in the following areas:

- **Hacking exercises:** Two projects that enable students to gain an understanding of the issues in intrusion detection and prevention.
- **Laboratory exercises:** A series of projects that involve programming and experimenting with concepts from the book.

- **Security education (SEED) projects:** The SEED projects are a set of hands-on exercises, or labs, covering a wide range of security topics.
- **Research projects:** A series of research assignments that instruct the students to research a particular topic on the Internet and write a report.
- **Programming projects:** A series of programming projects that cover a broad range of topics and that can be implemented in any suitable language on any platform.
- **Practical security assessments:** A set of exercises to examine current infrastructure and practices of an existing organization.
- **Firewall projects:** A portable network firewall visualization simulator is provided, together with exercises for teaching the fundamentals of firewalls.
- **Case studies:** A set of real-world case studies, including learning objectives, case description, and a series of case discussion questions.
- **Reading/report assignments:** A list of papers that can be assigned for reading and writing a report, plus suggested assignment wording.
- **Writing assignments:** A list of writing assignments to facilitate learning the material.
- **Webcasts for teaching computer security:** A catalog of webcast sites that can be used to enhance the course. An effective way of using this catalog is to select, or allow the student to select, one or a few videos to watch, and then to write a report/analysis of the video.

This diverse set of projects and other student exercises enables the instructor to use the book as one component in a rich and varied learning experience and to tailor a course plan to meet the specific needs of the instructor and students. See Appendix A in this book for details.

ACKNOWLEDGMENTS

This new edition has benefited from review by a number of people, who gave generously of their time and expertise. The following professors and instructors reviewed all or a large part of the manuscript: Bernardo Palazzi (Brown University), Jean Mayo (Michigan Technological University), Scott Kerlin (University of North Dakota), Philip Campbell (Ohio University), Scott Burgess (Humboldt State University), Stanley Wine (Hunter College/CUNY), and E. Mauricio Angee (Florida International University).

Thanks also to the many people who provided detailed technical reviews of one or more chapters: Umair Manzoor (UmZ), Adewumi Olatunji (FAGOSI Systems, Nigeria), Rob Meijer, Robin Goodchil, Greg Barnes (Involute Security LLC), Arturo Busleiman (Buanzo Consulting), Ryan M. Speers (Dartmouth College), Wynand van Staden (School of Computing, University of South Africa), Oh Sieng Chye, Michael Gromek, Samuel Weisberger, Brian Smithson (Ricoh Americas Corp, CISSP), Josef B. Weiss (CISSP), Robbert-Frank Ludwig (Veenendaal, ActStamp Information Security), William Perry, Daniela Zamfiroiu (CISSP), Rodrigo Ristow Branco, George Chetcuti (Technical Editor, TechGenix), Thomas Johnson (Director of Information Security at a banking holding company in Chicago, CISSP), Robert Yanus (CISSP), Rajiv Dasmohapatra (Wipro Ltd), Dirk Kotze, Ya'akov Yehudi, and Stanley Wine (Adjunct Lecturer, Computer Information Systems Department, Zicklin School of Business, Baruch College).

Dr. Lawrie Brown would first like to thank Bill Stallings for the pleasure of working with him to produce this text. I would also like to thank my colleagues in the School of Engineering and Information Technology, UNSW Canberra at the Australian Defence Force Academy for their encouragement and support. In particular, thanks to Gideon Creech, Edward Lewis, and Ben Whitham for discussion and review of some of the chapter content.

Finally, we would like to thank the many people responsible for the publication of the book, all of whom did their usual excellent job. This includes the staff at Pearson, particularly our editor Tracy Dunkelberger, her editorial assistant Kristy Alaura, and project manager Bob Engelhardt. Thanks also to the marketing and sales staffs at Pearson, without whose efforts this book would not be in front of you.

ACKNOWLEDGMENTS FOR THE GLOBAL EDITION

Pearson would like to thank and acknowledge Somitra Sanadhya (Indian Institute of Technology Ropar) for contributing to the Global Edition, and Arup Bhattacharya (RCC Institute of Technology), A. Kannammal (Coimbatore Institute of Technology), and Khyat Sharma for reviewing the Global Edition.

NOTATION

Symbol	Expression	Meaning
D, K	$D(K, Y)$	Symmetric decryption of ciphertext Y using secret key K
D, PR_a	$D(PR_a, Y)$	Asymmetric decryption of ciphertext Y using A's private key PR_a
D, PU_a	$D(PU_a, Y)$	Asymmetric decryption of ciphertext Y using A's public key PU_a
E, K	$E(K, X)$	Symmetric encryption of plaintext X using secret key K
E, PR_a	$E(PR_a, X)$	Asymmetric encryption of plaintext X using A's private key PR_a
E, PU_a	$E(PU_a, X)$	Asymmetric encryption of plaintext X using A's public key PU_a
K		Secret key
PR_a		Private key of user A
PU_a		Public key of user A
H	$H(X)$	Hash function of message X
$+$	$x + y$	Logical OR: x OR y
$\bullet$	$x \bullet y$	Logical AND: x AND y
$\sim$	$\sim x$	Logical NOT: NOT x
C		A characteristic formula, consisting of a logical formula over the values of attributes in a database
X	$X(C)$	Query set of C , the set of records satisfying C
$, X$	$ X(C) $	Magnitude of $X(C)$: the number of records in $X(C)$
$\cap$	$X(C) \cap X(D)$	Set intersection: the number of records in both $X(C)$ and $X(D)$
$ $	$x y$	x concatenated with y

ABOUT THE AUTHORS



Dr. William Stallings authored 18 textbooks, and, counting revised editions, a total of 70 books on various aspects of these subjects. His writings have appeared in numerous ACM and IEEE publications, including the *Proceedings of the IEEE and ACM Computing Reviews*. He has 13 times received the award for the best Computer Science textbook of the year from the Text and Academic Authors Association.

In over 30 years in the field, he has been a technical contributor, technical manager, and an executive with several high-technology firms. He has designed and implemented both TCP/IP-based and OSI-based protocol suites on a variety of computers and operating systems, ranging from microcomputers to mainframes. Currently he is an independent consultant whose clients have included computer and networking manufacturers and customers, software development firms, and leading-edge government research institutions.

He created and maintains the Computer Science Student Resource Site at ComputerScienceStudent.com. This site provides documents and links on a variety of subjects of general interest to computer science students (and professionals). He is a member of the editorial board of *Cryptologia*, a scholarly journal devoted to all aspects of cryptology.



Dr. Lawrie Brown is a visiting senior lecturer in the School of Engineering and Information Technology, UNSW Canberra at the Australian Defence Force Academy.

His professional interests include communications and computer systems security and cryptography, including research on pseudo-anonymous communication, authentication, security and trust issues in Web environments, the design of secure remote code execution environments using the functional language Erlang, and on the design and implementation of the LOKI family of block ciphers.

During his career, he has presented courses on cryptography, cybersecurity, data communications, data structures, and programming in Java to both undergraduate and postgraduate students.

OVERVIEW

1.1 Computer Security Concepts

- A Definition of Computer Security
- Examples
- The Challenges of Computer Security
- A Model for Computer Security

1.2 Threats, Attacks, and Assets

- Threats and Attacks
- Threats and Assets

1.3 Security Functional Requirements

1.4 Fundamental Security Design Principles

1.5 Attack Surfaces and Attack Trees

- Attack Surfaces
- Attack Trees

1.6 Computer Security Strategy

- Security Policy
- Security Implementation
- Assurance and Evaluation

1.7 Standards

1.8 Key Terms, Review Questions, and Problems

LEARNING OBJECTIVES

After studying this chapter, you should be able to:

- ◆ Describe the key security requirements of confidentiality, integrity, and availability.
- ◆ Discuss the types of security threats and attacks that must be dealt with and give examples of the types of threats and attacks that apply to different categories of computer and network assets.
- ◆ Summarize the functional requirements for computer security.
- ◆ Explain the fundamental security design principles.
- ◆ Discuss the use of attack surfaces and attack trees.
- ◆ Understand the principle aspects of a comprehensive security strategy.

This chapter provides an overview of computer security. We begin with a discussion of what we mean by computer security. In essence, computer security deals with computer-related assets that are subject to a variety of threats and for which various measures are taken to protect those assets. Accordingly, the next section of this chapter provides a brief overview of the categories of computer-related assets that users and system managers wish to preserve and protect, and a look at the various threats and attacks that can be made on those assets. Then, we survey the measures that can be taken to deal with such threats and attacks. This we do from three different viewpoints, in Sections 1.3 through 1.5. We then lay out in general terms a computer security strategy.

The focus of this chapter, and indeed this book, is on three fundamental questions:

1. What assets do we need to protect?
2. How are those assets threatened?
3. What can we do to counter those threats?

1.1 COMPUTER SECURITY CONCEPTS

A Definition of Computer Security

The NIST Internal/Interagency Report NISTIR 7298 (*Glossary of Key Information Security Terms*, May 2013) defines the term *computer security* as follows:

Computer Security: Measures and controls that ensure confidentiality, integrity, and availability of information system assets including hardware, software, firmware, and information being processed, stored, and communicated.

This definition introduces three key objectives that are at the heart of computer security:

- **Confidentiality:** This term covers two related concepts:
 - **Data confidentiality:**¹ Assures that private or confidential information is not made available or disclosed to unauthorized individuals.
 - **Privacy:** Assures that individuals control or influence what information related to them may be collected and stored and by whom and to whom that information may be disclosed.
- **Integrity:** This term covers two related concepts:
 - **Data integrity:** Assures that information and programs are changed only in a specified and authorized manner.
 - **System integrity:** Assures that a system performs its intended function in an unimpaired manner, free from deliberate or inadvertent unauthorized manipulation of the system.
- **Availability:** Assures that systems work promptly and service is not denied to authorized users.

These three concepts form what is often referred to as the **CIA triad**. The three concepts embody the fundamental security objectives for both data and for information and computing services. For example, the NIST standard FIPS 199 (*Standards for Security Categorization of Federal Information and Information Systems*, February 2004) lists confidentiality, integrity, and availability as the three security objectives for information and for information systems. FIPS 199 provides a useful characterization of these three objectives in terms of requirements and the definition of a loss of security in each category:

- **Confidentiality:** Preserving authorized restrictions on information access and disclosure, including means for protecting personal privacy and proprietary information. A loss of confidentiality is the unauthorized disclosure of information.
- **Integrity:** Guarding against improper information modification or destruction, including ensuring information nonrepudiation and authenticity. A loss of integrity is the unauthorized modification or destruction of information.
- **Availability:** Ensuring timely and reliable access to and use of information. A loss of availability is the disruption of access to or use of information or an information system.

Although the use of the CIA triad to define security objectives is well established, some in the security field feel that additional concepts are needed to present a complete picture (see Figure 1.1). Two of the most commonly mentioned are as follows:

- **Authenticity:** The property of being genuine and being able to be verified and trusted; confidence in the validity of a transmission, a message, or message

¹RFC 4949 (*Internet Security Glossary*, August 2007) defines *information* as “facts and ideas, which can be represented (encoded) as various forms of data,” and *data* as “information in a specific physical representation, usually a sequence of symbols that have meaning; especially a representation of information that can be processed or produced by a computer.” Security literature typically does not make much of a distinction; nor does this book.

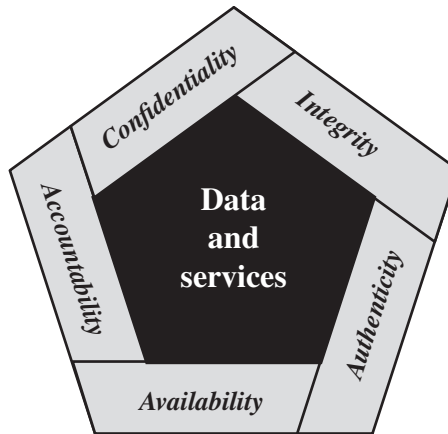


Figure 1.1 Essential Network and Computer Security Requirements

originator. This means verifying that users are who they say they are and that each input arriving at the system came from a trusted source.

- **Accountability:** The security goal that generates the requirement for actions of an entity to be traced uniquely to that entity. This supports nonrepudiation, deterrence, fault isolation, intrusion detection and prevention, and after-action recovery and legal action. Because truly secure systems are not yet an achievable goal, we must be able to trace a security breach to a responsible party. Systems must keep records of their activities to permit later forensic analysis to trace security breaches or to aid in transaction disputes.

Note that FIPS 199 includes authenticity under integrity.

Examples

We now provide some examples of applications that illustrate the requirements just enumerated.² For these examples, we use three levels of impact on organizations or individuals should there be a breach of security (i.e., a loss of confidentiality, integrity, or availability). These levels are defined in FIPS 199:

- **Low:** The loss could be expected to have a limited adverse effect on organizational operations, organizational assets, or individuals. A limited adverse effect means that, for example, the loss of confidentiality, integrity, or availability might: (i) cause a degradation in mission capability to an extent and duration that the organization is able to perform its primary functions, but the effectiveness of the functions is noticeably reduced; (ii) result in minor damage to organizational assets; (iii) result in minor financial loss; or (iv) result in minor harm to individuals.

²These examples are taken from a security policy document published by the Information Technology Security and Privacy Office at Purdue University.

- **Moderate:** The loss could be expected to have a serious adverse effect on organizational operations, organizational assets, or individuals. A serious adverse effect means that, for example, the loss might: (i) cause a significant degradation in mission capability to an extent and duration that the organization is able to perform its primary functions, but the effectiveness of the functions is significantly reduced; (ii) result in significant damage to organizational assets; (iii) result in significant financial loss; or (iv) result in significant harm to individuals that does not involve loss of life or serious life-threatening injuries.
- **High:** The loss could be expected to have a severe or catastrophic adverse effect on organizational operations, organizational assets, or individuals. A severe or catastrophic adverse effect means that, for example, the loss might: (i) cause a severe degradation in or loss of mission capability to an extent and duration that the organization is not able to perform one or more of its primary functions; (ii) result in major damage to organizational assets; (iii) result in major financial loss; or (iv) result in severe or catastrophic harm to individuals involving loss of life or serious life-threatening injuries.

CONFIDENTIALITY Student grade information is an asset whose confidentiality is considered to be highly important by students. In the United States, the release of such information is regulated by the Family Educational Rights and Privacy Act (FERPA). Grade information should only be available to students, their parents, and employees that require the information to do their job. Student enrollment information may have a moderate confidentiality rating. While still covered by FERPA, this information is seen by more people on a daily basis, is less likely to be targeted than grade information, and results in less damage if disclosed. Directory information, such as lists of students or faculty or departmental lists, may be assigned a low confidentiality rating or indeed no rating. This information is typically freely available to the public and published on a school's website.

INTEGRITY Several aspects of integrity are illustrated by the example of a hospital patient's allergy information stored in a database. The doctor should be able to trust that the information is correct and current. Now, suppose an employee (e.g., a nurse) who is authorized to view and update this information deliberately falsifies the data to cause harm to the hospital. The database needs to be restored to a trusted basis quickly, and it should be possible to trace the error back to the person responsible. Patient allergy information is an example of an asset with a high requirement for integrity. Inaccurate information could result in serious harm or death to a patient, and expose the hospital to massive liability.

An example of an asset that may be assigned a moderate level of integrity requirement is a website that offers a forum to registered users to discuss some specific topic. Either a registered user or a hacker could falsify some entries or deface the website. If the forum exists only for the enjoyment of the users, brings in little or no advertising revenue, and is not used for something important such as research, then potential damage is not severe. The Webmaster may experience some data, financial, and time loss.

An example of a low integrity requirement is an anonymous online poll. Many websites, such as news organizations, offer these polls to their users with very few

safeguards. However, the inaccuracy and unscientific nature of such polls is well understood.

AVAILABILITY The more critical a component or service is, the higher will be the level of availability required. Consider a system that provides authentication services for critical systems, applications, and devices. An interruption of service results in the inability for customers to access computing resources and staff to access the resources they need to perform critical tasks. The loss of the service translates into a large financial loss in lost employee productivity and potential customer loss.

An example of an asset that would typically be rated as having a moderate availability requirement is a public website for a university; the website provides information for current and prospective students and donors. Such a site is not a critical component of the university's information system, but its unavailability will cause some embarrassment.

An online telephone directory lookup application would be classified as a low availability requirement. Although the temporary loss of the application may be an annoyance, there are other ways to access the information, such as a hardcopy directory or the operator.

The Challenges of Computer Security

Computer security is both fascinating and complex. Some of the reasons are as follows:

1. Computer security is not as simple as it might first appear to the novice. The requirements seem to be straightforward; indeed, most of the major requirements for security services can be given self-explanatory one-word labels: confidentiality, authentication, nonrepudiation, and integrity. But the mechanisms used to meet those requirements can be quite complex, and understanding them may involve rather subtle reasoning.
2. In developing a particular security mechanism or algorithm, one must always consider potential attacks on those security features. In many cases, successful attacks are designed by looking at the problem in a completely different way, therefore exploiting an unexpected weakness in the mechanism.
3. Because of Point 2, the procedures used to provide particular services are often counterintuitive. Typically, a security mechanism is complex, and it is not obvious from the statement of a particular requirement that such elaborate measures are needed. Only when the various aspects of the threat are considered do elaborate security mechanisms make sense.
4. Having designed various security mechanisms, it is necessary to decide where to use them. This is true both in terms of physical placement (e.g., at what points in a network are certain security mechanisms needed) and in a logical sense [e.g., at what layer or layers of an architecture such as TCP/IP (Transmission Control Protocol/Internet Protocol) should mechanisms be placed].
5. Security mechanisms typically involve more than a particular algorithm or protocol. They also require that participants be in possession of some secret information (e.g., an encryption key), which raises questions about the creation, distribution, and protection of that secret information. There may also be a reliance on communications protocols whose behavior may complicate the task of

developing the security mechanism. For example, if the proper functioning of the security mechanism requires setting time limits on the transit time of a message from sender to receiver, then any protocol or network that introduces variable, unpredictable delays may render such time limits meaningless.

6. Computer security is essentially a battle of wits between a perpetrator who tries to find holes, and the designer or administrator who tries to close them. The great advantage that the attacker has is that he or she need only find a single weakness, while the designer must find and eliminate all weaknesses to achieve perfect security.
7. There is a natural tendency on the part of users and system managers to perceive little benefit from security investment until a security failure occurs.
8. Security requires regular, even constant monitoring, and this is difficult in today's short-term, overloaded environment.
9. Security is still too often an afterthought to be incorporated into a system after the design is complete, rather than being an integral part of the design process.
10. Many users and even security administrators view strong security as an impediment to efficient and user-friendly operation of an information system or use of information.

The difficulties just enumerated will be encountered in numerous ways as we examine the various security threats and mechanisms throughout this book.

A Model for Computer Security

We now introduce some terminology that will be useful throughout the book.³ Table 1.1 defines terms and Figure 1.2, based on [CCPS12a], shows the relationship among some of these terms. We start with the concept of a **system resource** or **asset**, that users and owners wish to protect. The assets of a computer system can be categorized as follows:

- **Hardware:** Including computer systems and other data processing, data storage, and data communications devices.
- **Software:** Including the operating system, system utilities, and applications.
- **Data:** Including files and databases, as well as security-related data, such as password files.
- **Communication facilities and networks:** Local and wide area network communication links, bridges, routers, and so on.

In the context of security, our concern is with the **vulnerabilities** of system resources. [NRC02] lists the following general categories of vulnerabilities of a computer system or network asset:

- The system can be **corrupted**, so it does the wrong thing or gives wrong answers. For example, stored data values may differ from what they should be because they have been improperly modified.

³See Chapter 0 for an explanation of RFCs.

Table 1.1 Computer Security Terminology**Adversary (threat agent)**

Individual, group, organization, or government that conducts or has the intent to conduct detrimental activities.

Attack

Any kind of malicious activity that attempts to collect, disrupt, deny, degrade, or destroy information system resources or the information itself.

Countermeasure

A device or techniques that has as its objective the impairment of the operational effectiveness of undesirable or adversarial activity, or the prevention of espionage, sabotage, theft, or unauthorized access to or use of sensitive information or information systems.

Risk

A measure of the extent to which an entity is threatened by a potential circumstance or event, and typically a function of 1) the adverse impacts that would arise if the circumstance or event occurs; and 2) the likelihood of occurrence.

Security Policy

A set of criteria for the provision of security services. It defines and constrains the activities of a data processing facility in order to maintain a condition of security for systems and data.

System Resource (Asset)

A major application, general support system, high impact program, physical plant, mission critical system, personnel, equipment, or a logically related group of systems.

Threat

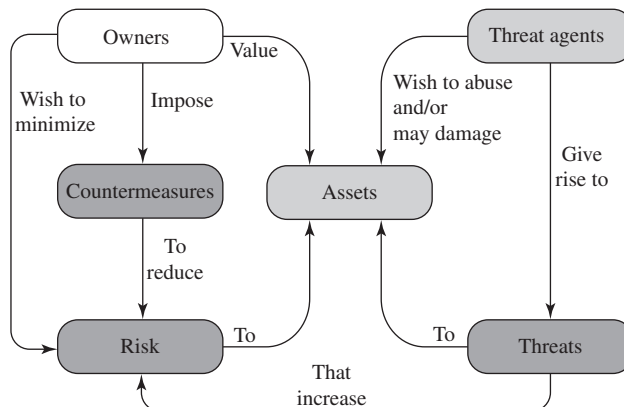
Any circumstance or event with the potential to adversely impact organizational operations (including mission, functions, image, or reputation), organizational assets, individuals, other organizations, or the Nation through an information system via unauthorized access, destruction, disclosure, modification of information, and/or denial of service.

Vulnerability

Weakness in an information system, system security procedures, internal controls, or implementation that could be exploited or triggered by a threat source.

Source: Stallings, William, Computer Security: Principles and Practice, 4e., ©2019. Reprinted and electronically reproduced by permission of pearson education, inc., new york, ny.

- The system can become **leaky**. For example, someone who should not have access to some or all of the information available through the network obtains such access.
- The system can become **unavailable** or very slow. That is, using the system or network becomes impossible or impractical.

**Figure 1.2 Security Concepts and Relationships**

These three general types of vulnerability correspond to the concepts of integrity, confidentiality, and availability, enumerated earlier in this section.

Corresponding to the various types of vulnerabilities to a system resource are **threats** that are capable of exploiting those vulnerabilities. A threat represents a potential security harm to an asset. An **attack** is a threat that is carried out (threat action) and, if successful, leads to an undesirable violation of security, or threat consequence. The agent carrying out the attack is referred to as an attacker or **threat agent**. We can distinguish two types of attacks:

- **Active attack:** An attempt to alter system resources or affect their operation.
- **Passive attack:** An attempt to learn or make use of information from the system that does not affect system resources.

We can also classify attacks based on the origin of the attack:

- **Inside attack:** Initiated by an entity inside the security perimeter (an “insider”). The insider is authorized to access system resources but uses them in a way not approved by those who granted the authorization.
- **Outside attack:** Initiated from outside the perimeter, by an unauthorized or illegitimate user of the system (an “outsider”). On the Internet, potential outside attackers range from amateur pranksters to organized criminals, international terrorists, and hostile governments.

Finally, a **countermeasure** is any means taken to deal with a security attack. Ideally, a countermeasure can be devised to **prevent** a particular type of attack from succeeding. When prevention is not possible, or fails in some instance, the goal is to **detect** the attack then **recover** from the effects of the attack. A countermeasure may itself introduce new vulnerabilities. In any case, residual vulnerabilities may remain after the imposition of countermeasures. Such vulnerabilities may be exploited by threat agents representing a residual level of **risk** to the assets. Owners will seek to minimize that risk given other constraints.

1.2 THREATS, ATTACKS, AND ASSETS

We now turn to a more detailed look at threats, attacks, and assets. First, we look at the types of security threats that must be dealt with, and then give some examples of the types of threats that apply to different categories of assets.

Threats and Attacks

Table 1.2, based on RFC 4949, describes four kinds of threat consequences and lists the kinds of attacks that result in each consequence.

Unauthorized disclosure is a threat to confidentiality. The following types of attacks can result in this threat consequence:

- **Exposure:** This can be deliberate, as when an insider intentionally releases sensitive information, such as credit card numbers, to an outsider. It can also be the result of a human, hardware, or software error, which results in an entity gaining unauthorized knowledge of sensitive data. There have been numerous

Table 1.2 Threat Consequences, and the Types of Threat Actions that Cause Each Consequence

Threat Consequence	Threat Action (Attack)
Unauthorized Disclosure A circumstance or event whereby an entity gains access to data for which the entity is not authorized.	Exposure: Sensitive data are directly released to an unauthorized entity. Interception: An unauthorized entity directly accesses sensitive data traveling between authorized sources and destinations. Inference: A threat action whereby an unauthorized entity indirectly accesses sensitive data (but not necessarily the data contained in the communication) by reasoning from characteristics or by-products of communications. Intrusion: An unauthorized entity gains access to sensitive data by circumventing a system's security protections.
Deception A circumstance or event that may result in an authorized entity receiving false data and believing it to be true.	Masquerade: An unauthorized entity gains access to a system or performs a malicious act by posing as an authorized entity. Falsification: False data deceive an authorized entity. Repudiation: An entity deceives another by falsely denying responsibility for an act.
Disruption A circumstance or event that interrupts or prevents the correct operation of system services and functions.	Incapacitation: Prevents or interrupts system operation by disabling a system component. Corruption: Undesirably alters system operation by adversely modifying system functions or data. Obstruction: A threat action that interrupts delivery of system services by hindering system operation.
Usurpation A circumstance or event that results in control of system services or functions by an unauthorized entity.	Misappropriation: An entity assumes unauthorized logical or physical control of a system resource. Misuse: Causes a system component to perform a function or service that is detrimental to system security.

Source: Based on RFC 4949

instances of this, such as universities accidentally posting confidential student information on the Web.

- **Interception:** Interception is a common attack in the context of communications. On a shared local area network (LAN), such as a wireless LAN or a broadcast Ethernet, any device attached to the LAN can receive a copy of packets intended for another device. On the Internet, a determined hacker can gain access to e-mail traffic and other data transfers. All of these situations create the potential for unauthorized access to data.
- **Inference:** An example of inference is known as traffic analysis, in which an adversary is able to gain information from observing the pattern of traffic on a network, such as the amount of traffic between particular pairs of hosts on the network. Another example is the inference of detailed information from a database by a user who has only limited access; this is accomplished by repeated queries whose combined results enable inference.
- **Intrusion:** An example of intrusion is an adversary gaining unauthorized access to sensitive data by overcoming the system's access control protections.

Deception is a threat to either system integrity or data integrity. The following types of attacks can result in this threat consequence:

- **Masquerade:** One example of masquerade is an attempt by an unauthorized user to gain access to a system by posing as an authorized user; this could happen if the unauthorized user has learned another user's logon ID and password. Another example is malicious logic, such as a Trojan horse, that appears to perform a useful or desirable function but actually gains unauthorized access to system resources, or tricks a user into executing other malicious logic.
- **Falsification:** This refers to the altering or replacing of valid data or the introduction of false data into a file or database. For example, a student may alter his or her grades on a school database.
- **Repudiation:** In this case, a user either denies sending data, or a user denies receiving or possessing the data.

Disruption is a threat to availability or system integrity. The following types of attacks can result in this threat consequence:

- **Incapacitation:** This is an attack on system availability. This could occur as a result of physical destruction of or damage to system hardware. More typically, malicious software, such as Trojan horses, viruses, or worms, could operate in such a way as to disable a system or some of its services.
- **Corruption:** This is an attack on system integrity. Malicious software in this context could operate in such a way that system resources or services function in an unintended manner. Or a user could gain unauthorized access to a system and modify some of its functions. An example of the latter is a user placing backdoor logic in the system to provide subsequent access to a system and its resources by other than the usual procedure.
- **Obstruction:** One way to obstruct system operation is to interfere with communications by disabling communication links or altering communication control information. Another way is to overload the system by placing excess burden on communication traffic or processing resources.

Usurpation is a threat to system integrity. The following types of attacks can result in this threat consequence:

- **Misappropriation:** This can include theft of service. An example is a distributed denial of service attack, when malicious software is installed on a number of hosts to be used as platforms to launch traffic at a target host. In this case, the malicious software makes unauthorized use of processor and operating system resources.
- **Misuse:** Misuse can occur by means of either malicious logic or a hacker that has gained unauthorized access to a system. In either case, security functions can be disabled or thwarted.

Threats and Assets

The assets of a computer system can be categorized as hardware, software, data, and communication lines and networks. In this subsection, we briefly describe these four

categories and relate these to the concepts of integrity, confidentiality, and availability introduced in Section 1.1 (see Figure 1.3 and Table 1.3).

HARDWARE A major threat to computer system hardware is the threat to availability. Hardware is the most vulnerable to attack and the least susceptible to automated controls. Threats include accidental and deliberate damage to equipment as well as theft. The proliferation of personal computers and workstations and the widespread use of LANs increase the potential for losses in this area. Theft of USB drives can lead to loss of confidentiality. Physical and administrative security measures are needed to deal with these threats.

SOFTWARE Software includes the operating system, utilities, and application programs. A key threat to software is an attack on availability. Software, especially application software, is often easy to delete. Software can also be altered or damaged to render it useless. Careful software configuration management, which includes making backups of the most recent version of software, can maintain high availability. A more difficult problem to deal with is software modification that results in a program that still functions but that behaves differently than before, which is a threat to integrity/authenticity. Computer viruses and related attacks fall into this category. A final problem is protection against software piracy. Although certain

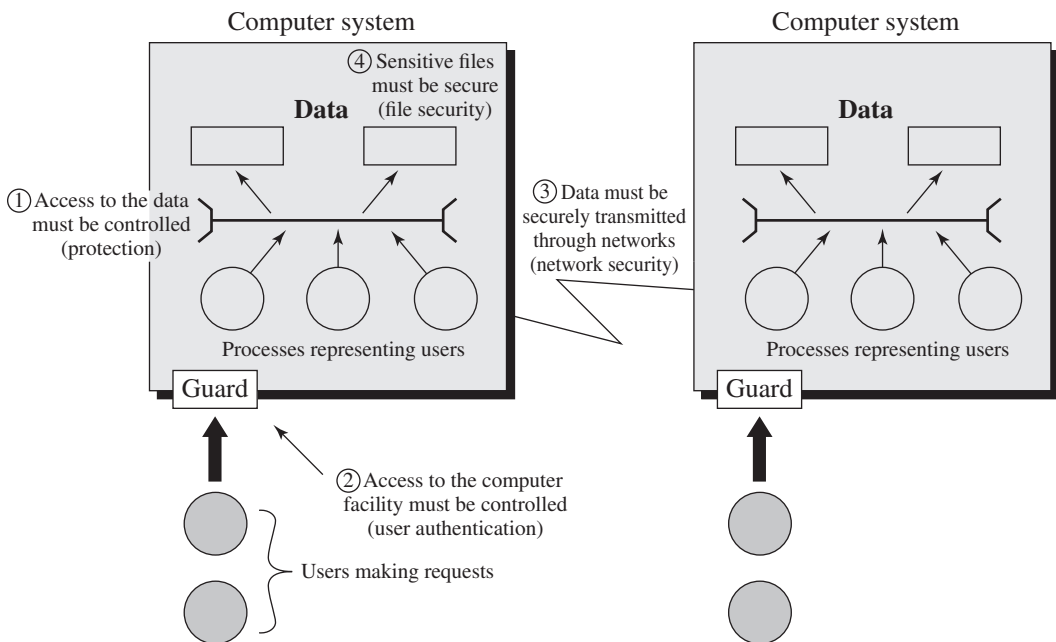


Figure 1.3 Scope of Computer Security

Note: This figure depicts security concerns other than physical security, including controlling of access to computers systems, safeguarding of data transmitted over communications systems, and safeguarding of stored data.

Table 1.3 Computer and Network Assets, with Examples of Threats

	Availability	Confidentiality	Integrity
Hardware	Equipment is stolen or disabled, thus denying service.	An unencrypted USB drive is stolen.	
Software	Programs are deleted, denying access to users.	An unauthorized copy of software is made.	A working program is modified, either to cause it to fail during execution or to cause it to do some unintended task.
Data	Files are deleted, denying access to users.	An unauthorized read of data is performed. An analysis of statistical data reveals underlying data.	Existing files are modified or new files are fabricated.
Communication Lines and Networks	Messages are destroyed or deleted. Communication lines or networks are rendered unavailable.	Messages are read. The traffic pattern of messages is observed.	Messages are modified, delayed, reordered, or duplicated. False messages are fabricated.

countermeasures are available, by and large the problem of unauthorized copying of software has not been solved.

DATA Hardware and software security are typically concerns of computing center professionals or individual concerns of personal computer users. A much more widespread problem is data security, which involves files and other forms of data controlled by individuals, groups, and business organizations.

Security concerns with respect to data are broad, encompassing availability, secrecy, and integrity. In the case of availability, the concern is with the destruction of data files, which can occur either accidentally or maliciously.

The obvious concern with secrecy is the unauthorized reading of data files or databases, and this area has been the subject of perhaps more research and effort than any other area of computer security. A less obvious threat to secrecy involves the analysis of data and manifests itself in the use of so-called statistical databases, which provide summary or aggregate information. Presumably, the existence of aggregate information does not threaten the privacy of the individuals involved. However, as the use of statistical databases grows, there is an increasing potential for disclosure of personal information. In essence, characteristics of constituent individuals may be identified through careful analysis. For example, if one table records the aggregate of the incomes of respondents A, B, C, and D and another records the aggregate of the incomes of A, B, C, D, and E, the difference between the two aggregates would be the income of E. This problem is exacerbated by the increasing desire to combine data sets. In many cases, matching several sets of data for consistency at different levels of aggregation requires access to individual units. Thus, the individual units, which are the subject of privacy concerns, are available at various stages in the processing of data sets.

Finally, data integrity is a major concern in most installations. Modifications to data files can have consequences ranging from minor to disastrous.

COMMUNICATION LINES AND NETWORKS Network security attacks can be classified as *passive attacks* and *active attacks*. A passive attack attempts to learn or make use of information from the system, but does not affect system resources. An active attack attempts to alter system resources or affect their operation.

Passive attacks are in the nature of eavesdropping on, or monitoring of, transmissions. The goal of the attacker is to obtain information that is being transmitted. Two types of passive attacks are the release of message contents and traffic analysis.

The **release of message contents** is easily understood. A telephone conversation, an electronic mail message, and a transferred file may contain sensitive or confidential information. We would like to prevent an opponent from learning the contents of these transmissions.

A second type of passive attack, **traffic analysis**, is more subtle. Suppose we had a way of masking the contents of messages or other information traffic so opponents, even if they captured the message, could not extract the information from the message. The common technique for masking contents is encryption. If we had encryption protection in place, an opponent might still be able to observe the pattern of these messages. The opponent could determine the location and identity of communicating hosts and could observe the frequency and length of messages being exchanged. This information might be useful in guessing the nature of the communication that was taking place.

Passive attacks are very difficult to detect because they do not involve any alteration of the data. Typically, the message traffic is sent and received in an apparently normal fashion and neither the sender nor receiver is aware that a third party has read the messages or observed the traffic pattern. However, it is feasible to prevent the success of these attacks, usually by means of encryption. Thus, the emphasis in dealing with passive attacks is on prevention rather than detection.

Active attacks involve some modification of the data stream or the creation of a false stream, and can be subdivided into four categories: replay, masquerade, modification of messages, and denial of service.

Replay involves the passive capture of a data unit and its subsequent retransmission to produce an unauthorized effect.

A **masquerade** takes place when one entity pretends to be a different entity. A masquerade attack usually includes one of the other forms of active attack. For example, authentication sequences can be captured and replayed after a valid authentication sequence has taken place, thus enabling an authorized entity with few privileges to obtain extra privileges by impersonating an entity that has those privileges.

Modification of messages simply means that some portion of a legitimate message is altered, or that messages are delayed or reordered, to produce an unauthorized effect. For example, a message stating, "Allow John Smith to read confidential file accounts" is modified to say, "Allow Fred Brown to read confidential file accounts."

The **denial of service** prevents or inhibits the normal use or management of communication facilities. This attack may have a specific target; for example, an entity may suppress all messages directed to a particular destination (e.g., the security

audit service). Another form of service denial is the disruption of an entire network, either by disabling the network or by overloading it with messages so as to degrade performance.

Active attacks present the opposite characteristics of passive attacks. Whereas passive attacks are difficult to detect, measures are available to prevent their success. On the other hand, it is quite difficult to prevent active attacks absolutely, because to do so would require physical protection of all communication facilities and paths at all times. Instead, the goal is to detect them and to recover from any disruption or delays caused by them. Because the detection has a deterrent effect, it may also contribute to prevention.

1.4 FUNDAMENTAL SECURITY DESIGN PRINCIPLES

Despite years of research and development, it has not been possible to develop security design and implementation techniques that systematically exclude security flaws and prevent all unauthorized actions. In the absence of such foolproof techniques, it is useful to have a set of widely agreed design principles that can guide the development of protection mechanisms. The National Centers of Academic Excellence in Information Assurance/Cyber Defense, which is jointly sponsored by the U.S. National Security Agency and the U. S. Department of Homeland Security, list the following as fundamental security design principles [NCAE13]:

- Economy of mechanism
- Fail-safe defaults
- Complete mediation
- Open design

- Separation of privilege
- Least privilege
- Least common mechanism
- Psychological acceptability
- Isolation
- Encapsulation
- Modularity
- Layering
- Least astonishment

The first eight listed principles were first proposed in [SALT75] and have withstood the test of time. In this section, we briefly discuss each principle.

Economy of mechanism means the design of security measures embodied in both hardware and software should be as simple and small as possible. The motivation for this principle is that relatively simple, small design is easier to test and verify thoroughly. With a complex design, there are many more opportunities for an adversary to discover subtle weaknesses to exploit that may be difficult to spot ahead of time. The more complex the mechanism is, the more likely it is to possess exploitable flaws. Simple mechanisms tend to have fewer exploitable flaws and require less maintenance. Furthermore, because configuration management issues are simplified, updating or replacing a simple mechanism becomes a less intensive process. In practice, this is perhaps the most difficult principle to honor. There is a constant demand for new features in both hardware and software, complicating the security design task. The best that can be done is to keep this principle in mind during system design to try to eliminate unnecessary complexity.

Fail-safe default means access decisions should be based on permission rather than exclusion. That is, the default situation is lack of access, and the protection scheme identifies conditions under which access is permitted. This approach exhibits a better failure mode than the alternative approach, where the default is to permit access. A design or implementation mistake in a mechanism that gives explicit permission tends to fail by refusing permission, a safe situation that can be quickly detected. On the other hand, a design or implementation mistake in a mechanism that explicitly excludes access tends to fail by allowing access, a failure that may long go unnoticed in normal use. For example, most file access systems work on this principle and virtually all protected services on client/server systems work this way.

Complete mediation means every access must be checked against the access control mechanism. Systems should not rely on access decisions retrieved from a cache. In a system designed to operate continuously, this principle requires that, if access decisions are remembered for future use, careful consideration be given to how changes in authority are propagated into such local memories. File access systems appear to provide an example of a system that complies with this principle. However, typically, once a user has opened a file, no check is made to see if permissions change. To fully implement complete mediation, every time a user reads a field or record in a file, or a data item in a database, the system must exercise access control. This resource-intensive approach is rarely used.

Open design means the design of a security mechanism should be open rather than secret. For example, although encryption keys must be secret, encryption algorithms should be open to public scrutiny. The algorithms can then be reviewed by many experts, and users can therefore have high confidence in them. This is the philosophy behind the National Institute of Standards and Technology (NIST) program of standardizing encryption and hash algorithms, and has led to the widespread adoption of NIST-approved algorithms.

Separation of privilege is defined in [SALT75] as a practice in which multiple privilege attributes are required to achieve access to a restricted resource. A good example of this is multifactor user authentication, which requires the use of multiple techniques, such as a password and a smart card, to authorize a user. The term is also now applied to any technique in which a program is divided into parts that are limited to the specific privileges they require in order to perform a specific task. This is used to mitigate the potential damage of a computer security attack. One example of this latter interpretation of the principle is removing high privilege operations to another process and running that process with the higher privileges required to perform its tasks. Day-to-day interfaces are executed in a lower privileged process.

Least privilege means every process and every user of the system should operate using the least set of privileges necessary to perform the task. A good example of the use of this principle is role-based access control, as will be described in Chapter 4. The system security policy can identify and define the various roles of users or processes. Each role is assigned only those permissions needed to perform its functions. Each permission specifies a permitted access to a particular resource (such as read and write access to a specified file or directory, and connect access to a given host and port). Unless permission is granted explicitly, the user or process should not be able to access the protected resource. More generally, any access control system should allow each user only the privileges that are authorized for that user. There is also a temporal aspect to the least privilege principle. For example, system programs or administrators who have special privileges should have those privileges only when necessary; when they are doing ordinary activities the privileges should be withdrawn. Leaving them in place just opens the door to accidents.

Least common mechanism means the design should minimize the functions shared by different users, providing mutual security. This principle helps reduce the number of unintended communication paths and reduces the amount of hardware and software on which all users depend, thus making it easier to verify if there are any undesirable security implications.

Psychological acceptability implies the security mechanisms should not interfere unduly with the work of users, and at the same time meet the needs of those who authorize access. If security mechanisms hinder the usability or accessibility of resources, users may opt to turn off those mechanisms. Where possible, security mechanisms should be transparent to the users of the system or at most introduce minimal obstruction. In addition to not being intrusive or burdensome, security procedures must reflect the user's mental model of protection. If the protection procedures do not make sense to the user or if the user must translate his or her image of protection into a substantially different protocol, the user is likely to make errors.

Isolation is a principle that applies in three contexts. First, public access systems should be isolated from critical resources (data, processes, etc.) to prevent disclosure or tampering. In cases where the sensitivity or criticality of the information is high, organizations may want to limit the number of systems on which that data are stored and isolate them, either physically or logically. Physical isolation may include ensuring that no physical connection exists between an organization's public access information resources and an organization's critical information. When implementing logical isolation solutions, layers of security services and mechanisms should be established between public systems and secure systems that is responsible for protecting critical resources. Second, the processes and files of individual users should be isolated from one another except where it is explicitly desired. All modern operating systems provide facilities for such isolation, so individual users have separate, isolated process space, memory space, and file space, with protections for preventing unauthorized access. And finally, security mechanisms should be isolated in the sense of preventing access to those mechanisms. For example, logical access control may provide a means of isolating cryptographic software from other parts of the host system and for protecting cryptographic software from tampering and the keys from replacement or disclosure.

Encapsulation can be viewed as a specific form of isolation based on object-oriented functionality. Protection is provided by encapsulating a collection of procedures and data objects in a domain of its own so that the internal structure of a data object is accessible only to the procedures of the protected subsystem and the procedures may be called only at designated domain entry points.

Modularity in the context of security refers both to the development of security functions as separate, protected modules, and to the use of a modular architecture for mechanism design and implementation. With respect to the use of separate security modules, the design goal here is to provide common security functions and services, such as cryptographic functions, as common modules. For example, numerous protocols and applications make use of cryptographic functions. Rather than implementing such functions in each protocol or application, a more secure design is provided by developing a common cryptographic module that can be invoked by numerous protocols and applications. The design and implementation effort can then focus on the secure design and implementation of a single cryptographic module, including mechanisms to protect the module from tampering. With respect to the use of a modular architecture, each security mechanism should be able to support migration to new technology or upgrade of new features without requiring an entire system redesign. The security design should be modular so that individual parts of the security design can be upgraded without the requirement to modify the entire system.

Layering refers to the use of multiple, overlapping protection approaches addressing the people, technology, and operational aspects of information systems. By using multiple, overlapping protection approaches, the failure or circumvention of any individual protection approach will not leave the system unprotected. We will see throughout this book that a layering approach is often used to provide multiple barriers between an adversary and protected information or services. This technique is often referred to as *defense in depth*.

Least astonishment means a program or user interface should always respond in the way that is least likely to astonish the user. For example, the mechanism for authorization should be transparent enough to a user that the user has a good intuitive understanding of how the security goals map to the provided security mechanism.

1.5 ATTACK SURFACES AND ATTACK TREES

Section 1.2 provided an overview of the spectrum of security threats and attacks facing computer and network systems. Section 8.1 will go into more detail about the nature of attacks and the types of adversaries that present security threats. In this section, we elaborate on two concepts that are useful in evaluating and classifying threats: attack surfaces and attack trees.

Attack Surfaces

An attack surface consists of the reachable and exploitable vulnerabilities in a system [BELL16, MANA11, HOWA03]. Examples of attack surfaces are the following:

- Open ports on outward facing Web and other servers, and code listening on those ports
- Services available on the inside of a firewall
- Code that processes incoming data, e-mail, XML, office documents, and industry-specific custom data exchange formats
- Interfaces, SQL, and web forms
- An employee with access to sensitive information vulnerable to a social engineering attack

Attack surfaces can be categorized in the following way:

- **Network attack surface:** This category refers to vulnerabilities over an enterprise network, wide-area network, or the Internet. Included in this category are network protocol vulnerabilities, such as those used for a denial-of-service attack, disruption of communications links, and various forms of intruder attacks.
- **Software attack surface:** This refers to vulnerabilities in application, utility, or operating system code. A particular focus in this category is Web server software.
- **Human attack surface:** This category refers to vulnerabilities created by personnel or outsiders, such as social engineering, human error, and trusted insiders.

An attack surface analysis is a useful technique for assessing the scale and severity of threats to a system. A systematic analysis of points of vulnerability makes developers and security analysts aware of where security mechanisms are required. Once an attack surface is defined, designers may be able to find ways to make the surface smaller, thus making the task of the adversary more difficult. The attack surface also provides guidance on setting priorities for testing, strengthening security measures, or modifying the service or application.

As illustrated in Figure 1.4, the use of layering, or defense in depth, and attack surface reduction complement each other in mitigating security risk.

Attack Trees

An attack tree is a branching, hierarchical data structure that represents a set of potential techniques for exploiting security vulnerabilities [MAUW05, MOOR01, SCHN99]. The security incident that is the goal of the attack is represented as the root node of the tree, and the ways by which an attacker could reach that goal are iteratively and incrementally represented as branches and subnodes of the tree. Each subnode defines a subgoal, and each subgoal may have its own set of further subgoals, and so on. The final nodes on the paths outward from the root, that is, the leaf nodes, represent different ways to initiate an attack. Each node other than a leaf is either an AND-node or an OR-node. To achieve the goal represented by an AND-node, the subgoals represented by all of that node's subnodes must be achieved; and for an OR-node, at least one of the subgoals must be achieved. Branches can be labeled with values representing difficulty, cost, or other attack attributes, so that alternative attacks can be compared.

The motivation for the use of attack trees is to effectively exploit the information available on attack patterns. Organizations such as CERT publish security advisories that have enabled the development of a body of knowledge about both general attack strategies and specific attack patterns. Security analysts can use the attack tree to document security attacks in a structured form that reveals key vulnerabilities. The attack tree can guide both the design of systems and applications, and the choice and strength of countermeasures.

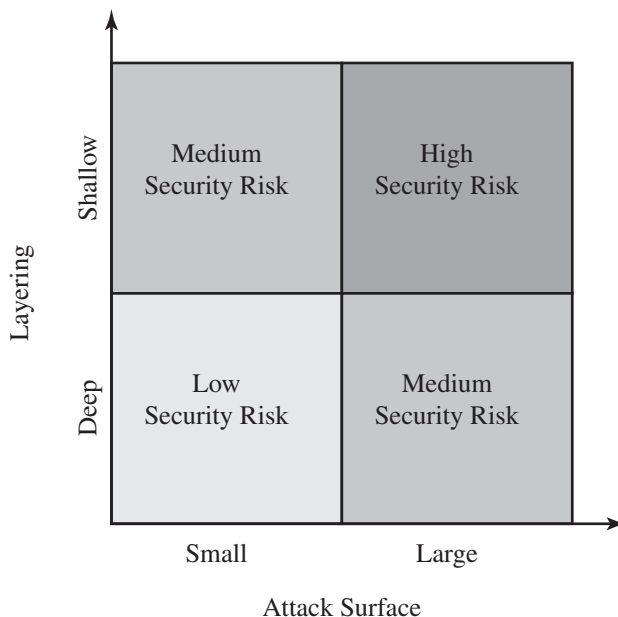


Figure 1.4 Defense in Depth and Attack Surface

Figure 1.5, based on a figure in [DIMI07], is an example of an attack tree analysis for an Internet banking authentication application. The root of the tree is the objective of the attacker, which is to compromise a user's account. The shaded boxes on the tree are the leaf nodes, which represent events that comprise the attacks. The white boxes are categories which consist of one or more specific attack events (leaf nodes). Note that in this tree, all the nodes other than leaf nodes are OR-nodes. The analysis used to generate this tree considered the three components involved in authentication:

- **User terminal and user (UT/U):** These attacks target the user equipment, including the tokens that may be involved, such as smartcards or other password generators, as well as the actions of the user.
- **Communications channel (CC):** This type of attack focuses on communication links.
- **Internet banking server (IBS):** These types of attacks are offline attack against the servers that host the Internet banking application.

Five overall attack strategies can be identified, each of which exploits one or more of the three components. The five strategies are as follows:

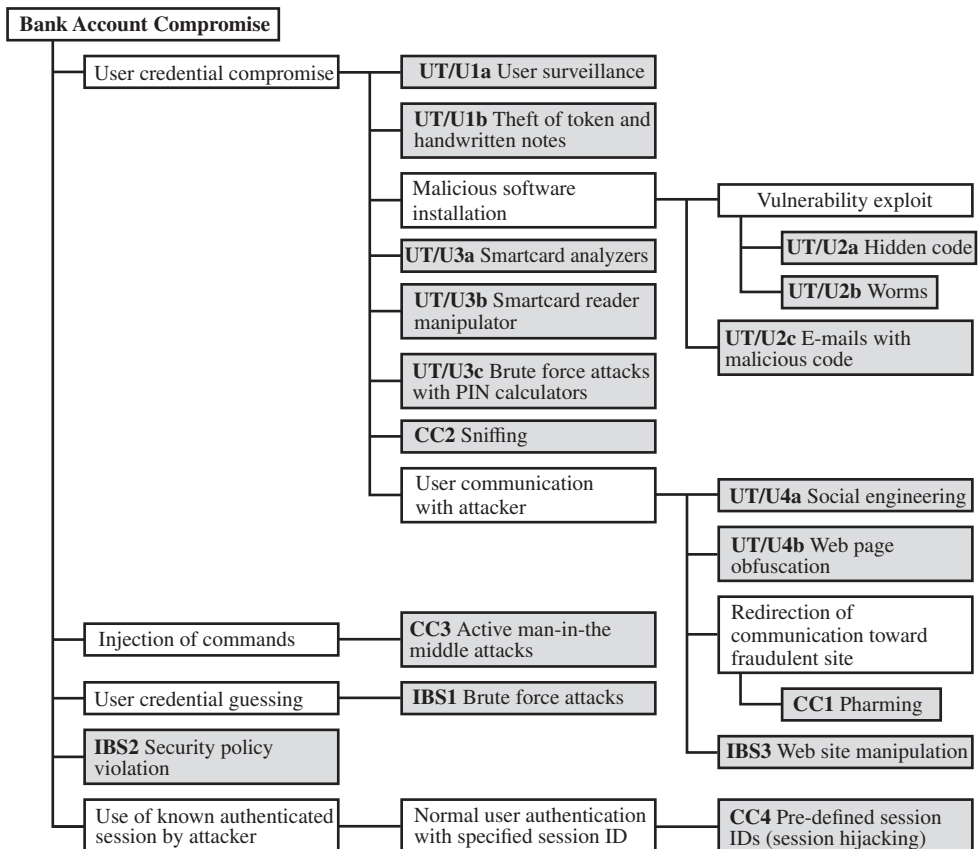


Figure 1.5 An Attack Tree for Internet Banking Authentication

- **User credential compromise:** This strategy can be used against many elements of the attack surface. There are procedural attacks, such as monitoring a user's action to observe a PIN or other credential, or theft of the user's token or handwritten notes. An adversary may also compromise token information using a variety of token attack tools, such as hacking the smartcard or using a brute force approach to guess the PIN. Another possible strategy is to embed malicious software to compromise the user's login and password. An adversary may also attempt to obtain credential information via the communication channel (sniffing). Finally, an adversary may use various means to engage in communication with the target user, as shown in Figure 1.5.
- **Injection of commands:** In this type of attack, the attacker is able to intercept communication between the UT and the IBS. Various schemes can be used to be able to impersonate the valid user and so gain access to the banking system.
- **User credential guessing:** It is reported in [HILT06] that brute force attacks against some banking authentication schemes are feasible by sending random usernames and passwords. The attack mechanism is based on distributed zombie personal computers, hosting automated programs for username- or password-based calculation.
- **Security policy violation:** For example, violating the bank's security policy in combination with weak access control and logging mechanisms, an employee may cause an internal security incident and expose a customer's account.
- **Use of known authenticated session:** This type of attack persuades or forces the user to connect to the IBS with a preset session ID. Once the user authenticates to the server, the attacker may utilize the known session ID to send packets to the IBS, spoofing the user's identity.

Figure 1.5 provides a thorough view of the different types of attacks on an Internet banking authentication application. Using this tree as a starting point, security analysts can assess the risk of each attack and, using the design principles outlined in the preceding section, design a comprehensive security facility. [DIMO07] provides a good account of the results of this design effort.

1.6 COMPUTER SECURITY STRATEGY

We conclude this chapter with a brief look at the overall strategy for providing computer security. [LAMP04] suggests that a comprehensive security strategy involves three aspects:

- **Specification/policy:** What is the security scheme supposed to do?
- **Implementation/mechanisms:** How does it do it?
- **Correctness/assurance:** Does it really work?

Security Policy

The first step in devising security services and mechanisms is to develop a security policy. Those involved with computer security use the term *security policy* in various ways. At the least, a security policy is an informal description of desired system

behavior [NRC91]. Such informal policies may reference requirements for security, integrity, and availability. More usefully, a security policy is a formal statement of rules and practices that specify or regulate how a system or organization provides security services to protect sensitive and critical system resources (RFC 4949). Such a formal security policy lends itself to being enforced by the system's technical controls as well as its management and operational controls.

In developing a security policy, a security manager needs to consider the following factors:

- The value of the assets being protected
- The vulnerabilities of the system
- Potential threats and the likelihood of attacks

Further, the manager must consider the following trade-offs:

- **Ease of use versus security:** Virtually all security measures involve some penalty in the area of ease of use. The following are some examples: Access control mechanisms require users to remember passwords and perhaps perform other access control actions. Firewalls and other network security measures may reduce available transmission capacity or slow response time. Virus-checking software reduces available processing power and introduces the possibility of system crashes or malfunctions due to improper interaction between the security software and the operating system.
- **Cost of security versus cost of failure and recovery:** In addition to ease of use and performance costs, there are direct monetary costs in implementing and maintaining security measures. All of these costs must be balanced against the cost of security failure and recovery if certain security measures are lacking. The cost of security failure and recovery must take into account not only the value of the assets being protected and the damages resulting from a security violation, but also the risk, which is the probability that a particular threat will exploit a particular vulnerability with a particular harmful result.

Security policy is thus a business decision, possibly influenced by legal requirements.

Security Implementation

Security implementation involves four complementary courses of action:

- **Prevention:** An ideal security scheme is one in which no attack is successful. Although this is not practical in all cases, there is a wide range of threats in which prevention is a reasonable goal. For example, consider the transmission of encrypted data. If a secure encryption algorithm is used, and if measures are in place to prevent unauthorized access to encryption keys, then attacks on confidentiality of the transmitted data will be prevented.
- **Detection:** In a number of cases, absolute protection is not feasible, but it is practical to detect security attacks. For example, there are intrusion detection systems designed to detect the presence of unauthorized individuals logged onto a system. Another example is detection of a denial of service attack,

in which communications or processing resources are consumed so they are unavailable to legitimate users.

- **Response:** If security mechanisms detect an ongoing attack, such as a denial of service attack, the system may be able to respond in such a way as to halt the attack and prevent further damage.
- **Recovery:** An example of recovery is the use of backup systems, so if data integrity is compromised, a prior, correct copy of the data can be reloaded.

Assurance and Evaluation

Those who are “consumers” of computer security services and mechanisms (e.g., system managers, vendors, customers, and end users) desire a belief that the security measures in place work as intended. That is, security consumers want to feel that the security infrastructure of their systems meet security requirements and enforce security policies. These considerations bring us to the concepts of assurance and evaluation.

Assurance is an attribute of an information system that provides grounds for having confidence that the system operates such that the system’s security policy is enforced. This encompasses both system design and system implementation. Thus, assurance deals with the questions, “Does the security system design meet its requirements?” and “Does the security system implementation meet its specifications?” Assurance is expressed as a degree of confidence, not in terms of a formal proof that a design or implementation is correct. The state of the art in proving designs and implementations is such that it is not possible to provide absolute proof. Much work has been done in developing formal models that define requirements and characterize designs and implementations, together with logical and mathematical techniques for addressing these issues. But assurance is still a matter of degree.

Evaluation is the process of examining a computer product or system with respect to certain criteria. Evaluation involves testing and may also involve formal analytic or mathematical techniques. The central thrust of work in this area is the development of evaluation criteria that can be applied to any security system (encompassing security services and mechanisms) and that are broadly supported for making product comparisons.

1.8 KEY TERMS, REVIEW QUESTIONS, AND PROBLEMS

Key Terms

access control	data confidentiality	interceptions
active attack	data integrity	intrusion
adversary	denial of service	isolation
asset	disruption	layering
assurance	economy of mechanism	least astonishment
attack	encapsulation	least common mechanism
attack surface	encryption	least privilege
attack tree	evaluation	masquerade
authentication	exposure	misappropriation
authenticity	fail-safe defaults	misuse
availability	falsification	modularity
complete mediation	incapacitation	nonrepudiation
confidentiality	inference	obstruction
corruption	inside attack	open design
countermeasure	integrity	OSI security architecture

(Continued)

outside attack passive attack prevent privacy psychological acceptability replay repudiation	risk security attack security mechanism security policy security service separation of privilege system integrity	system resource threat agent traffic analysis unauthorized disclosure usurpation vulnerabilities
--	---	---

Review Questions

- 1.1 What is meant by the CIA triad?
- 1.2 What is the difference between data integrity and system integrity?
- 1.3 List and briefly define the kinds of threat consequences and the types of threat actions which cause these consequences.
- 1.4 List and briefly define the fundamental security design principles.
- 1.5 What is a security policy? What are the actions involved when implementing a security policy?
- 1.6 Differentiate between a network attack surface and a software attack surface.

Problems

- 1.1 Consider a student information system (SIS) in which students provide a university student number (USN) and a card for account access. Give examples of confidentiality, integrity, and availability requirements associated with the system and, in each case, indicate the degree of the importance of the requirement.
- 1.2 Repeat Problem 1.1 for a network routing system that routes data packets through a network based on the IP address provided by the sender.
- 1.3 Consider a desktop publishing system used to produce documents for various organizations.
 - a. Give an example of a type of publication for which confidentiality of the stored data is the most important requirement.
 - b. Give an example of a type of publication in which data integrity is the most important requirement.
 - c. Give an example in which system availability is the most important requirement.
- 1.4 For each of the following assets, assign a low, moderate, or high impact level for the loss of confidentiality, availability, and integrity, respectively. Justify your answers.
 - a. An organization managing public information on its Web server.
 - b. A law enforcement organization managing extremely sensitive investigative information.
 - c. A financial organization managing routine administrative information (not privacy-related information).
 - d. An information system used for large acquisitions in a contracting organization contains both sensitive, pre-solicitation phase contract information and routine administrative information. Assess the impact for the two data sets separately and the information system as a whole.
 - e. A power plant contains a SCADA (supervisory control and data acquisition) system controlling the distribution of electric power for a large military installation. The SCADA system contains both real-time sensor data and routine administrative information. Assess the impact for the two data sets separately and the information system as a whole.

- 1.5** Consider the following general code for allowing access to a resource:

```
DWORD dwRet = IsAccessAllowed(...);
if (dwRet == ERROR_ACCESS_DENIED) {
    // Security check failed.
    // Inform user that access is denied.
} else {
    // Security check OK.
}
```

a. Explain the security flaw in this program.

b. Rewrite the code to avoid the flaw.

Hint: Consider the design principle of fail-safe defaults.

- 1.6** Develop an attack tree for gaining access to the contents of a physical safe.
- 1.7** Consider a company whose operations are housed in two buildings on the same property: one building is headquarters, the other building contains network and computer services. The property is physically protected by a fence around the perimeter. The only entrance to the property is through a guarded front gate. The local networks are split between the Headquarters' LAN and the Network Services' LAN. Internet users connect to the Web server through a firewall. Dial-up users get access to a particular server on the Network Services' LAN. Develop an attack tree in which the root node represents disclosure of proprietary secrets. Include physical, social engineering, and technical attacks. The tree may contain both AND and OR nodes. Develop a tree that has at least 15 leaf nodes.
- 1.8** Read all of the classic papers cited in the Recommended Reading document at <http://williamstallings.com/ComputerSecurity/>. Compose a 500–1000 word paper (or 8–12 slide presentation) that summarizes the key concepts that emerge from these papers, emphasizing concepts that are common to most or all of the papers.

PART ONE: Computer Security Technology and Principles

CHAPTER

2

CRYPTOGRAPHIC TOOLS

2.1 Confidentiality with Symmetric Encryption

- Symmetric Encryption
- Symmetric Block Encryption Algorithms
- Stream Ciphers

2.2 Message Authentication and Hash Functions

- Authentication Using Symmetric Encryption
- Message Authentication without Message Encryption
- Secure Hash Functions
- Other Applications of Hash Functions

2.3 Public-Key Encryption

- Public-Key Encryption Structure
- Applications for Public-Key Cryptosystems
- Requirements for Public-Key Cryptography
- Asymmetric Encryption Algorithms

2.4 Digital Signatures and Key Management

- Digital Signature
- Public-Key Certificates
- Symmetric Key Exchange Using Public-Key Encryption
- Digital Envelopes

2.5 Random and Pseudorandom Numbers

- The Use of Random Numbers
- Random versus Pseudorandom

2.6 Practical Application: Encryption of Stored Data

2.7 Key Terms, Review Questions, and Problems

LEARNING OBJECTIVES

After studying this chapter, you should be able to:

- ◆ Explain the basic operation of symmetric block encryption algorithms.
- ◆ Compare and contrast block encryption and stream encryption.
- ◆ Discuss the use of secure hash functions for message authentication.
- ◆ List other applications of secure hash functions.
- ◆ Explain the basic operation of asymmetric block encryption algorithms.
- ◆ Present an overview of the digital signature mechanism and explain the concept of digital envelopes.
- ◆ Explain the significance of random and pseudorandom numbers in cryptography.

An important element in many computer security services and applications is the use of cryptographic algorithms. This chapter provides an overview of the various types of algorithms, together with a discussion of their applicability. For each type of algorithm, we will introduce the most important standardized algorithms in common use. For the technical details of the algorithms themselves, see Part Four.

We begin with symmetric encryption, which is used in the widest variety of contexts, primarily to provide confidentiality. Next, we examine secure hash functions and discuss their use in message authentication. The next section examines public-key encryption, also known as asymmetric encryption. We then discuss the two most important applications of public-key encryption, namely digital signatures and key management. In the case of digital signatures, asymmetric encryption and secure hash functions are combined to produce an extremely useful tool.

Finally, in this chapter, we provide an example of an application area for cryptographic algorithms by looking at the encryption of stored data.

2.1 CONFIDENTIALITY WITH SYMMETRIC ENCRYPTION

The universal technique for providing confidentiality for transmitted or stored data is symmetric encryption. This section introduces the basic concept of symmetric encryption. This is followed by an overview of the two most important symmetric encryption algorithms: the Data Encryption Standard (DES) and the Advanced Encryption Standard (AES), which are block encryption algorithms. Finally, this section introduces the concept of symmetric stream encryption algorithms.

Symmetric Encryption

Symmetric encryption, also referred to as conventional encryption or single-key encryption, was the only type of encryption in use prior to the introduction of public-key encryption in the late 1970s. Countless individuals and groups, from Julius Caesar to the German U-boat force to present-day diplomatic, military, and commercial users,

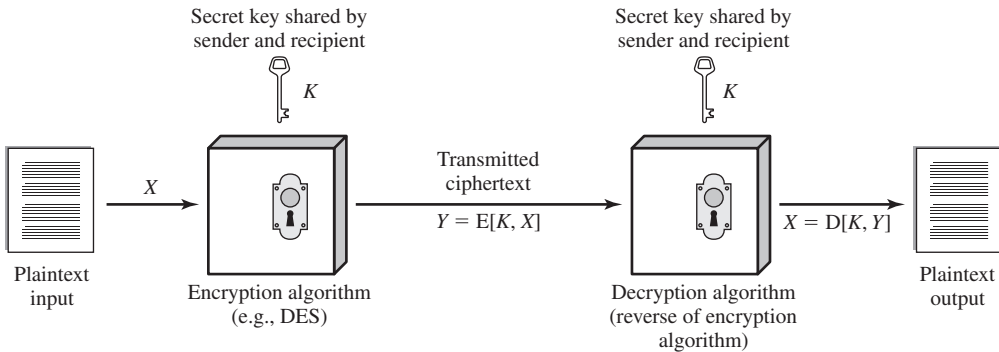


Figure 2.1 Simplified Model of Symmetric Encryption

have used symmetric encryption for secret communication. It remains the more widely used of the two types of encryption.

A symmetric encryption scheme has five ingredients (see Figure 2.1):

- **Plaintext:** This is the original message or data that is fed into the algorithm as input.
- **Encryption algorithm:** The encryption algorithm performs various substitutions and transformations on the plaintext.
- **Secret key:** The secret key is also input to the encryption algorithm. The exact substitutions and transformations performed by the algorithm depend on the key.
- **Ciphertext:** This is the scrambled message produced as output. It depends on the plaintext and the secret key. For a given message, two different keys will produce two different ciphertexts.
- **Decryption algorithm:** This is essentially the encryption algorithm run in reverse. It takes the ciphertext and the secret key and produces the original plaintext.

There are two requirements for secure use of symmetric encryption:

1. We need a strong encryption algorithm. At a minimum, we would like the algorithm to be such that an opponent who knows the algorithm and has access to one or more ciphertexts would be unable to decipher the ciphertext or figure out the key. This requirement is usually stated in a stronger form: The opponent should be unable to decrypt ciphertext or discover the key even if he or she is in possession of a number of ciphertexts together with the plaintext that produced each ciphertext.
2. The sender and receiver must have obtained copies of the secret key in a secure fashion and must keep the key secure. If someone can discover the key and knows the algorithm, all communication using this key is readable.

There are two general approaches to attacking a symmetric encryption scheme. The first attack is known as **cryptanalysis**. Cryptanalytic attacks rely on the nature of the algorithm plus perhaps some knowledge of the general characteristics of the plaintext, or even some sample plaintext-ciphertext pairs. This type of attack exploits the characteristics of the algorithm to attempt to deduce a specific plaintext or to

deduce the key being used. If the attack succeeds in deducing the key, the effect is catastrophic: All future and past messages encrypted with that key are compromised.

The second method, known as the **brute-force attack**, is to try every possible key on a piece of ciphertext until an intelligible translation into plaintext is obtained. On average, half of all possible keys must be tried to achieve success. That is, if there are x different keys, on average an attacker would discover the actual key after $x/2$ tries. There is more to a brute-force attack than simply running through all possible keys. Unless known plaintext is provided, the analyst must be able to recognize plaintext as plaintext. If the message is just plain text in English, then the result pops out easily, although the task of recognizing English would have to be automated. If the text message has been compressed before encryption, then recognition is more difficult. And if the message is some more general type of data, such as a numerical file, and this has been compressed, the problem becomes even more difficult to automate. Thus, to supplement the brute-force approach, some degree of knowledge about the expected plaintext is needed, and some means of automatically distinguishing plaintext from garble is also needed.

Symmetric Block Encryption Algorithms

The most commonly used symmetric encryption algorithms are block ciphers. A block cipher processes the plaintext input in fixed-size blocks and produces a block of ciphertext of equal size for each plaintext block. The algorithm processes longer plaintext amounts as a series of fixed-size blocks. The most important symmetric algorithms, all of which are block ciphers, are the Data Encryption Standard (DES), triple DES, and the Advanced Encryption Standard (AES); see Table 2.1. This subsection provides an overview of these algorithms. Chapter 20 will present the technical details.

DATA ENCRYPTION STANDARD Until recently, the most widely used encryption scheme was based on the Data Encryption Standard (DES) adopted in 1977 by the National Bureau of Standards, now the National Institute of Standards and Technology (NIST), as FIPS PUB 46 (*Data Encryption Standard*, January 1977).¹ The algorithm itself is referred to as the Data Encryption Algorithm (DEA). DES takes a plaintext block of 64 bits and a key of 56 bits, to produce a ciphertext block of 64 bits.

Concerns about the strength of DES fall into two categories: concerns about the algorithm itself, and concerns about the use of a 56-bit key. The first concern refers to

Table 2.1 Comparison of Three Popular Symmetric Encryption Algorithms

	DES	Triple DES	AES
Plaintext block size (bits)	64	64	128
Ciphertext block size (bits)	64	64	128
Key size (bits)	56	112 or 168	128, 192, or 256

DES = Data Encryption Standard

AES = Advanced Encryption Standard

¹See Appendix C for more information on NIST and similar organizations, and the “List of NIST and ISO Documents” for related publications that we discuss.

the possibility that cryptanalysis is possible by exploiting the characteristics of the DES algorithm. Over the years, there have been numerous attempts to find and exploit weaknesses in the algorithm, making DES the most-studied encryption algorithm in existence. Despite numerous approaches, no one has so far reported a fatal weakness in DES.

A more serious concern is key length. With a key length of 56 bits, there are 2^{56} possible keys, which is approximately 7.2×10^{16} keys. Given the speed of commercial off-the-shelf processors, this key length is woefully inadequate. A paper from Seagate Technology [SEAG08] suggests that a rate of one billion (10^9) key combinations per second is reasonable for today's multicore computers. Recent offerings confirm this. Both Intel and AMD now offer hardware-based instructions to accelerate the use of AES. Tests run on a contemporary multicore Intel machine resulted in an encryption rate of about half a billion encryptions per second [BASU12]. Another recent analysis suggests that with contemporary supercomputer technology, a rate of 10^{13} encryptions/s is reasonable [AROR12].

With these results in mind, Table 2.2 shows how much time is required for a brute-force attack for various key sizes. As can be seen, a single PC can break DES in about a year; if multiple PCs work in parallel, the time is drastically shortened. And today's supercomputers should be able to find a key in about an hour. Key sizes of 128 bits or greater are effectively unbreakable using simply a brute-force approach. Even if we managed to speed up the attacking system by a factor of 1 trillion (10^{12}), it would still take over 100,000 years to break a code using a 128-bit key.

Fortunately, there are a number of alternatives to DES, the most important of which are triple DES and AES, discussed in the remainder of this section.

TRIPLE DES The life of DES was extended by the use of triple DES (3DES), which involves repeating the basic DES algorithm three times, using either two or three unique keys, for a key size of 112 or 168 bits. 3DES was first standardized for use in financial applications in ANSI standard X9.17 in 1985. 3DES was incorporated as part of the Data Encryption Standard in 1999, with the publication of FIPS PUB 46-3.

3DES has two attractions that assure its widespread use over the next few years. First, with its 168-bit key length, it overcomes the vulnerability to brute-force attack of DES. Second, the underlying encryption algorithm in 3DES is the same as in DES. This algorithm has been subjected to more scrutiny than any other encryption algorithm over a longer period of time, and no effective cryptanalytic attack based on the algorithm rather than brute force has been found. Accordingly, there is a high

Table 2.2 Average Time Required for Exhaustive Key Search

Key Size (bits)	Cipher	Number of Alternative Keys	Time Required at 10^9 decryptions/ μ s	Time Required at 10^{13} decryptions/ μ s
56	DES	$2^{56} \approx 7.2 \times 10^{16}$	$2^{55} \mu\text{s} = 1.125 \text{ years}$	1 hour
128	AES	$2^{128} \approx 3.4 \times 10^{38}$	$2^{127} \mu\text{s} = 5.3 \times 10^{21} \text{ years}$	$5.3 \times 10^{17} \text{ years}$
168	Triple DES	$2^{168} \approx 3.7 \times 10^{50}$	$2^{167} \mu\text{s} = 5.8 \times 10^{33} \text{ years}$	$5.8 \times 10^{29} \text{ years}$
192	AES	$2^{192} \approx 6.3 \times 10^{57}$	$2^{191} \mu\text{s} = 9.8 \times 10^{40} \text{ years}$	$9.8 \times 10^{36} \text{ years}$
256	AES	$2^{256} \approx 1.2 \times 10^{77}$	$2^{255} \mu\text{s} = 1.8 \times 10^{60} \text{ years}$	$1.8 \times 10^{56} \text{ years}$

level of confidence that 3DES is very resistant to cryptanalysis. If security were the only consideration, then 3DES would be an appropriate choice for a standardized encryption algorithm for decades to come.

The principal drawback of 3DES is that the algorithm is relatively sluggish in software. The original DES was designed for mid-1970s hardware implementation and does not produce efficient software code. 3DES, which requires three times as many calculations as DES, is correspondingly slower. A secondary drawback is that both DES and 3DES use a 64-bit block size. For reasons of both efficiency and security, a larger block size is desirable.

ADVANCED ENCRYPTION STANDARD Because of its drawbacks, 3DES is not a reasonable candidate for long-term use. As a replacement, NIST in 1997 issued a call for proposals for a new Advanced Encryption Standard (AES), which should have a security strength equal to or better than 3DES and significantly improved efficiency. In addition to these general requirements, NIST specified that AES must be a symmetric block cipher with a block length of 128 bits and support for key lengths of 128, 192, and 256 bits. Evaluation criteria included security, computational efficiency, memory requirements, hardware and software suitability, and flexibility.

In a first round of evaluation, 15 proposed algorithms were accepted. A second round narrowed the field to 5 algorithms. NIST completed its evaluation process and published the final standard as FIPS PUB 197 (*Advanced Encryption Standard*, November 2001). NIST selected Rijndael as the proposed AES algorithm. AES is now widely available in commercial products. AES will be described in detail in Chapter 20.

PRACTICAL SECURITY ISSUES Typically, symmetric encryption is applied to a unit of data larger than a single 64-bit or 128-bit block. E-mail messages, network packets, database records, and other plaintext sources must be broken up into a series of fixed-length block for encryption by a symmetric block cipher. The simplest approach to multiple-block encryption is known as electronic codebook (ECB) mode, in which plaintext is handled b bits at a time and each block of plaintext is encrypted using the same key. Typically $b = 64$ or $b = 128$. Figure 2.2a shows the ECB mode. A plain text of length nb is divided into n b -bit blocks ($P_1, P_2, \dots, P_n$). Each block is encrypted using the same algorithm and the same encryption key, to produce a sequence of n b -bit blocks of ciphertext ($C_1, C_2, \dots, C_n$).

For lengthy messages, the ECB mode may not be secure. A cryptanalyst may be able to exploit regularities in the plaintext to ease the task of decryption. For example, if it is known that the message always starts out with certain predefined fields, then the cryptanalyst may have a number of known plaintext-ciphertext pairs with which to work.

To increase the security of symmetric block encryption for large sequences of data, a number of alternative techniques have been developed, called **modes of operation**. These modes overcome the weaknesses of ECB; each mode has its own particular advantages. This topic will be explored in Chapter 20.

Stream Ciphers

A *block cipher* processes the input one block of elements at a time, producing an output block for each input block. A *stream cipher* processes the input elements continuously, producing output one element at a time, as it goes along. Although block

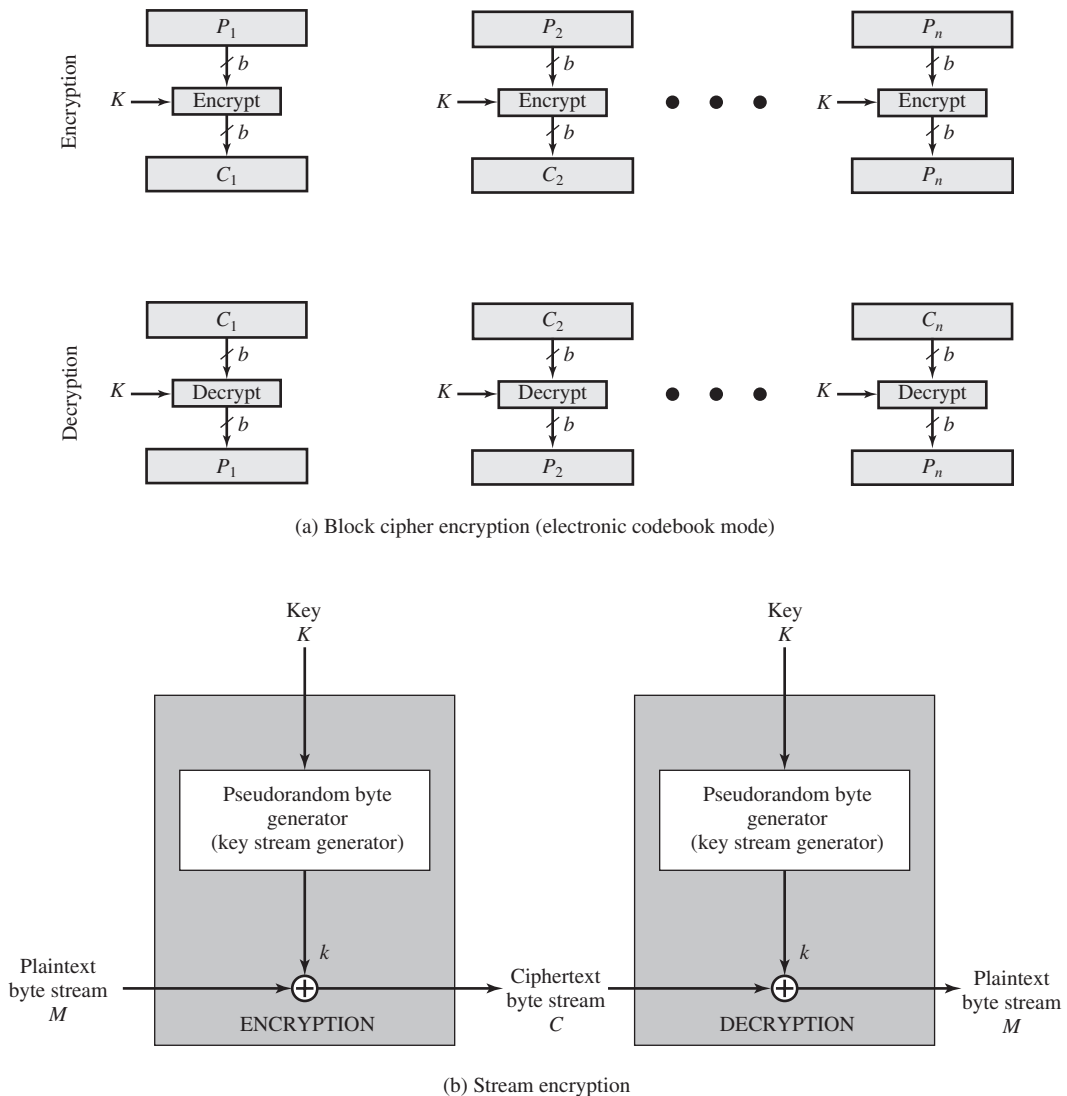


Figure 2.2 Types of Symmetric Encryption

ciphers are far more common, there are certain applications in which a stream cipher is more appropriate. Examples will be given subsequently in this book.

A typical stream cipher encrypts plaintext one byte at a time, although a stream cipher may be designed to operate on one bit at a time or on units larger than a byte at a time. Figure 2.2b is a representative diagram of stream cipher structure. In this structure, a key is input to a pseudorandom bit generator that produces a stream of 8-bit numbers that are apparently random. A pseudorandom stream is one that is unpredictable without knowledge of the input key and which has an apparently random character (see Section 2.5). The output of the generator, called a **keystream**,

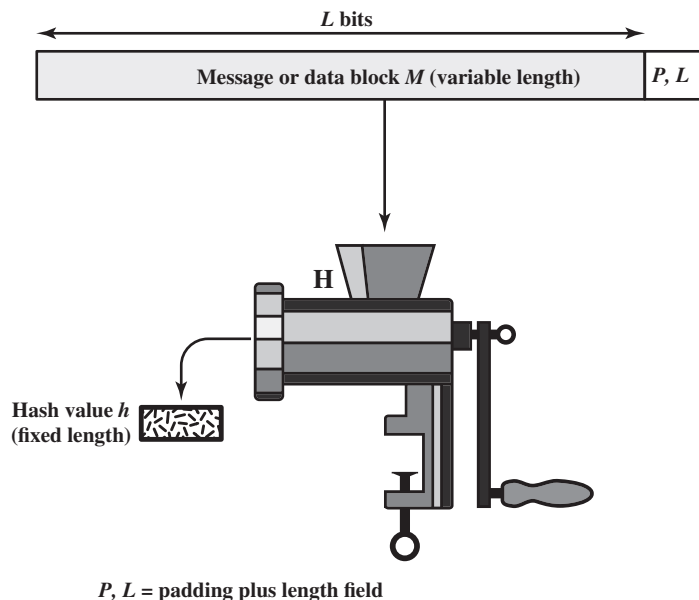
is combined one byte at a time with the plaintext stream using the bitwise exclusive-OR (XOR) operation.

With a properly designed pseudorandom number generator, a stream cipher can be as secure as a block cipher of comparable key length. The primary advantage of a stream cipher is that stream ciphers are almost always faster and use far less code than do block ciphers. The advantage of a block cipher is that you can reuse keys. For applications that require encryption/decryption of a stream of data, such as over a data communications channel or a browser/Web link, a stream cipher might be the better alternative. For applications that deal with blocks of data, such as file transfer, e-mail, and database, block ciphers may be more appropriate. However, either type of cipher can be used in virtually any application.

2.2 MESSAGE AUTHENTICATION AND HASH FUNCTIONS

ONE-WAY HASH FUNCTION An alternative to the message authentication code is the one-way hash function. As with the message authentication code, a hash function accepts a variable-size message M as input and produces a fixed-size message digest $H(M)$ as output (see Figure 2.4). Typically, the message is padded out to an integer multiple of some fixed length (e.g., 1024 bits) and the padding includes the value of the length of the original message in bits. The length field is a security measure to increase the difficulty for an attacker to produce an alternative message with the same hash value.

Unlike the MAC, a hash function does not take a secret key as input. Figure 2.5 illustrates three ways in which the message can be authenticated using a hash function. The message digest can be encrypted using symmetric encryption



P, L = padding plus length field

Figure 2.4 Cryptographic Hash Function; $h = H(M)$

⁴Recall from our discussion of practical security issues in Section 2.1 that for large amounts of data, some mode of operation is needed to apply a block cipher such as DES to amounts of data larger than a single block. For the MAC application mentioned here, DES is applied in what is known as cipher block chaining mode (CBC). In essence, DES is applied to each 64-bit block of the message in sequence, with the input to the encryption algorithm being the XOR of the current plaintext block and the preceding ciphertext block. The MAC is derived from the final block encryption. See Chapter 20 for a discussion of CBC.

(see Figure 2.5a); if it is assumed that only the sender and receiver share the encryption key, then authenticity is assured. The message digest can also be encrypted using public-key encryption (see Figure 2.5b); this is explained in Section 2.3. The public-key approach has two advantages: It provides a digital signature as well as message authentication, and it does not require the distribution of keys to communicating parties.

These two approaches have an advantage over approaches that encrypt the entire message, in that less computation is required. But an even more common approach is the use of a technique that avoids encryption altogether. Several reasons for this interest are pointed out in [TSUD92]:

- Encryption software is quite slow. Even though the amount of data to be encrypted per message is small, there may be a steady stream of messages into and out of a system.
- Encryption hardware costs are nonnegligible. Low-cost chip implementations of DES and AES are available, but the cost adds up if all nodes in a network must have this capability.
- Encryption hardware is optimized toward large data sizes. For small blocks of data, a high proportion of the time is spent in initialization/invoke overhead.
- An encryption algorithm may be protected by a patent.

Figure 2.5c shows a technique that uses a hash function but no encryption for message authentication. This technique, known as a keyed hash MAC, assumes that two communicating parties, say A and B, share a common secret key K . This secret key is incorporated into the process of generating a hash code. In the approach illustrated in Figure 2.5c, when A has a message to send to B, it calculates the hash function over the concatenation of the secret key and the message: $MD_M = H(K \| M \| K)$.⁵ It then sends $[M \| MD_M]$ to B. Because B possesses K , it can recompute $H(K \| M \| K)$ and verify MD_M . Because the secret key itself is not sent, it should not be possible for an attacker to modify an intercepted message. As long as the secret key remains secret, it should not be possible for an attacker to generate a false message.

Note the secret key is used as both a prefix and a suffix to the message. If the secret key is used as either only a prefix or only a suffix, the scheme is less secure. This topic will be discussed in Chapter 21. Chapter 21 also describes a scheme known as HMAC, which is somewhat more complex than the approach of Figure 2.5c and which has become the standard approach for a keyed hash MAC.

Secure Hash Functions

The one-way hash function, or secure hash function, is important not only in message authentication but also in digital signatures. In this section, we begin with a discussion of requirements for a secure hash function. Then we discuss specific algorithms.

HASH FUNCTION REQUIREMENTS The purpose of a hash function is to produce a “fingerprint” of a file, message, or other block of data. To be useful for message authentication, a hash function H must have the following properties:

1. H can be applied to a block of data of any size.
2. H produces a fixed-length output.
3. $H(x)$ is relatively easy to compute for any given x , making both hardware and software implementations practical.

⁵ $\|$ denotes concatenation.

4. For any given code h , it is computationally infeasible to find x such that $H(x) = h$. A hash function with this property is referred to as **one-way** or **preimage resistant**.⁶
5. For any given block x , it is computationally infeasible to find $y \neq x$ with $H(y) = H(x)$. A hash function with this property is referred to as **second preimage resistant**. This is sometimes referred to as **weak collision resistant**.
6. It is computationally infeasible to find any pair (x, y) such that $H(x) = H(y)$. A hash function with this property is referred to as **collision resistant**. This is sometimes referred to as **strong collision resistant**.

The first three properties are requirements for the practical application of a hash function to message authentication.

The fourth property is the one-way property: It is easy to generate a code given a message, but virtually impossible to generate a message given a code. This property is important if the authentication technique involves the use of a secret value (see Figure 2.5c). The secret value itself is not sent; however, if the hash function is not one-way, an attacker can easily discover the secret value: If the attacker can observe or intercept a transmission, the attacker obtains the message M and the hash code $MD_M = H(K \| M \| K)$. The attacker then inverts the hash function to obtain $K \| M \| K = H^{-1}(MD_M)$. Because the attacker now has both M and $(K \| M \| K)$ it is a trivial matter to recover K .

The fifth property guarantees that it is impossible to find an alternative message with the same hash value as a given message. This prevents forgery when an encrypted hash code is used (see Figure 2.5a and b). If this property were not true, an attacker would be capable of the following sequence: First, observe or intercept a message plus its encrypted hash code; second, generate an unencrypted hash code from the message; and third, generate an alternate message with the same hash code.

A hash function that satisfies the first five properties in the preceding list is referred to as a weak hash function. If the sixth property is also satisfied, then it is referred to as a strong hash function. A strong hash function protects against an attack in which one party generates a message for another party to sign. For example, suppose Alice agrees to sign an IOU for a small amount that is sent to her by Bob. Suppose also that Bob can find two messages with the same hash value, one of which requires Alice to pay the small amount, and one that requires a large payment. Alice signs the first message, and Bob is then able to claim that the second message is authentic.

In addition to providing authentication, a message digest also provides data integrity. It performs the same function as a frame check sequence: If any bits in the message are accidentally altered in transit, the message digest will be in error.

SECURITY OF HASH FUNCTIONS As with symmetric encryption, there are two approaches to attacking a secure hash function: cryptanalysis and brute-force attack. As with symmetric encryption algorithms, cryptanalysis of a hash function involves exploiting logical weaknesses in the algorithm.

⁶For $f(x) = y$, x is said to be a preimage of y . Unless f is one-to-one, there may be multiple preimage values for a given y .

The strength of a hash function against brute-force attacks depends solely on the length of the hash code produced by the algorithm. For a hash code of length n , the level of effort required is proportional to the following:

Preimage resistant	2^n
Second preimage resistant	2^n
Collision resistant	$2^{n/2}$

If collision resistance is required (and this is desirable for a general-purpose secure hash code), then the value $2^{n/2}$ determines the strength of the hash code against brute-force attacks. Van Oorschot and Wiener [VANO94] presented a design for a \$10 million collision search machine for MD5, which has a 128-bit hash length, that could find a collision in 24 days. Thus, a 128-bit code may be viewed as inadequate. The next step up, if a hash code is treated as a sequence of 32 bits, is a 160-bit hash length. With a hash length of 160 bits, the same search machine would require over four thousand years to find a collision. With today's technology, the time would be much shorter, so 160 bits now appears suspect.

SECURE HASH FUNCTION ALGORITHMS In recent years, the most widely used hash function has been the Secure Hash Algorithm (SHA). SHA was developed by the National Institute of Standards and Technology (NIST) and published as a federal information processing standard (FIPS 180) in 1993. When weaknesses were discovered in SHA, a revised version was issued as FIPS 180-1 in 1995 and is generally referred to as SHA-1. SHA-1 produces a hash value of 160 bits. In 2002, NIST produced a revised version of the standard, FIPS 180-2, that defined three new versions of SHA, with hash value lengths of 256, 384, and 512 bits, known as SHA-256, SHA-384, and SHA-512. These new versions, collectively known as SHA-2, have the same underlying structure and use the same types of modular arithmetic and logical binary operations as SHA-1. SHA-2, particularly the 512-bit version, would appear to provide unassailable security. However, because of the structural similarity of SHA-2 to SHA-1, NIST decided to standardize a new hash function that is very different from SHA-2 and SHA-1. This new hash function, known as SHA-3, was published in 2015 and is now available as an alternative to SHA-2.

Other Applications of Hash Functions

We have discussed the use of hash functions for message authentication and for the creation of digital signatures (the latter will be discussed in more detail later in this chapter). Here are two other examples of secure hash function applications:

- **Passwords:** Chapter 3 will explain a scheme in which a hash of a password is stored by an operating system rather than the password itself. Thus, the actual password is not retrievable by a hacker who gains access to the password file. In simple terms, when a user enters a password, the hash of that password is compared to the stored hash value for verification. This application requires preimage resistance and perhaps second preimage resistance.

- **Intrusion detection:** Store the hash value for a file, $H(F)$, for each file on a system and secure the hash values (e.g., on a write-locked drive or write-once optical disk that is kept secure). One can later determine if a file has been modified by recomputing $H(F)$. An intruder would need to change F without changing $H(F)$. This application requires weak second preimage resistance.

2.3 PUBLIC-KEY ENCRYPTION

Of equal importance to symmetric encryption is public-key encryption, which finds use in message authentication and key distribution.

Public-Key Encryption Structure

Public-key encryption, first publicly proposed by Diffie and Hellman in 1976 [DIFF76], is the first truly revolutionary advance in encryption in literally thousands of years. Public-key algorithms are based on mathematical functions rather than on simple operations on bit patterns, such as are used in symmetric encryption algorithms. More important, public-key cryptography is **asymmetric**, involving the use of two separate keys, in contrast to symmetric encryption, which uses only one key. The use of two keys has profound consequences in the areas of confidentiality, key distribution, and authentication.

Before proceeding, we should first mention several common misconceptions concerning public-key encryption. One is that public-key encryption is more secure from cryptanalysis than symmetric encryption. In fact, the security of any encryption scheme depends on (1) the length of the key and (2) the computational work involved in breaking a cipher. There is nothing in principle about either symmetric or public-key encryption that makes one superior to another from the point of view of resisting cryptanalysis. A second misconception is that public-key encryption is a general-purpose technique that has made symmetric encryption obsolete. On the contrary, because of the computational overhead of current public-key encryption schemes, there seems no foreseeable likelihood that symmetric encryption will be abandoned. Finally, there is a feeling that key distribution is trivial when using public-key encryption, compared to the rather cumbersome handshaking involved with key distribution centers for symmetric encryption. For public-key key distribution, some form of protocol is needed, often involving a central agent, and the procedures involved are no simpler or any more efficient than those required for symmetric encryption.

A public-key encryption scheme has six ingredients (see Figure 2.6a):

- **Plaintext:** This is the readable message or data that is fed into the algorithm as input.
- **Encryption algorithm:** The encryption algorithm performs various transformations on the plaintext.
- **Public and private key:** This is a pair of keys that have been selected so if one is used for encryption, the other is used for decryption. The exact transformations

performed by the encryption algorithm depend on the public or private key that is provided as input.⁷

- **Ciphertext:** This is the scrambled message produced as output. It depends on the plaintext and the key. For a given message, two different keys will produce two different ciphertexts.
- **Decryption algorithm:** This algorithm accepts the ciphertext and the matching key and produces the original plaintext.

As the names suggest, the public key of the pair is made public for others to use, while the private key is known only to its owner. A general-purpose public-key cryptographic algorithm relies on one key for encryption and a different but related key for decryption.

The essential steps are the following:

1. Each user generates a pair of keys to be used for the encryption and decryption of messages.
2. Each user places one of the two keys in a public register or other accessible file. This is the public key. The companion key is kept private. As Figure 2.6a suggests, each user maintains a collection of public keys obtained from others.
3. If Bob wishes to send a private message to Alice, Bob encrypts the message using Alice's public key.
4. When Alice receives the message, she decrypts it using her private key. No other recipient can decrypt the message because only Alice knows Alice's private key.

With this approach, all participants have access to public keys, and private keys are generated locally by each participant and therefore need never be distributed. As long as a user protects his or her private key, incoming communication is secure. At any time, a user can change the private key and publish the companion public key to replace the old public key.

Figure 2.6b illustrates another mode of operation of public-key cryptography. In this scheme, a user encrypts data using his or her own private key. Anyone who knows the corresponding public key will then be able to decrypt the message.

Note the scheme of Figure 2.6a is directed toward providing **confidentiality**. Only the intended recipient should be able to decrypt the ciphertext because only the intended recipient is in possession of the required private key. Whether in fact confidentiality is provided depends on a number of factors, including the security of the algorithm, whether the private key is kept secure, and the security of any protocol of which the encryption function is a part.

The scheme of Figure 2.6b is directed toward providing **authentication** and/or **data integrity**. If a user is able to successfully recover the plaintext from Bob's ciphertext using Bob's public key, this indicates only Bob could have encrypted the

⁷The key used in symmetric encryption is typically referred to as a **secret key**. The two keys used for public-key encryption are referred to as the **public key** and the **private key**. Invariably, the private key is kept secret, but it is referred to as a private key rather than a secret key to avoid confusion with symmetric encryption.

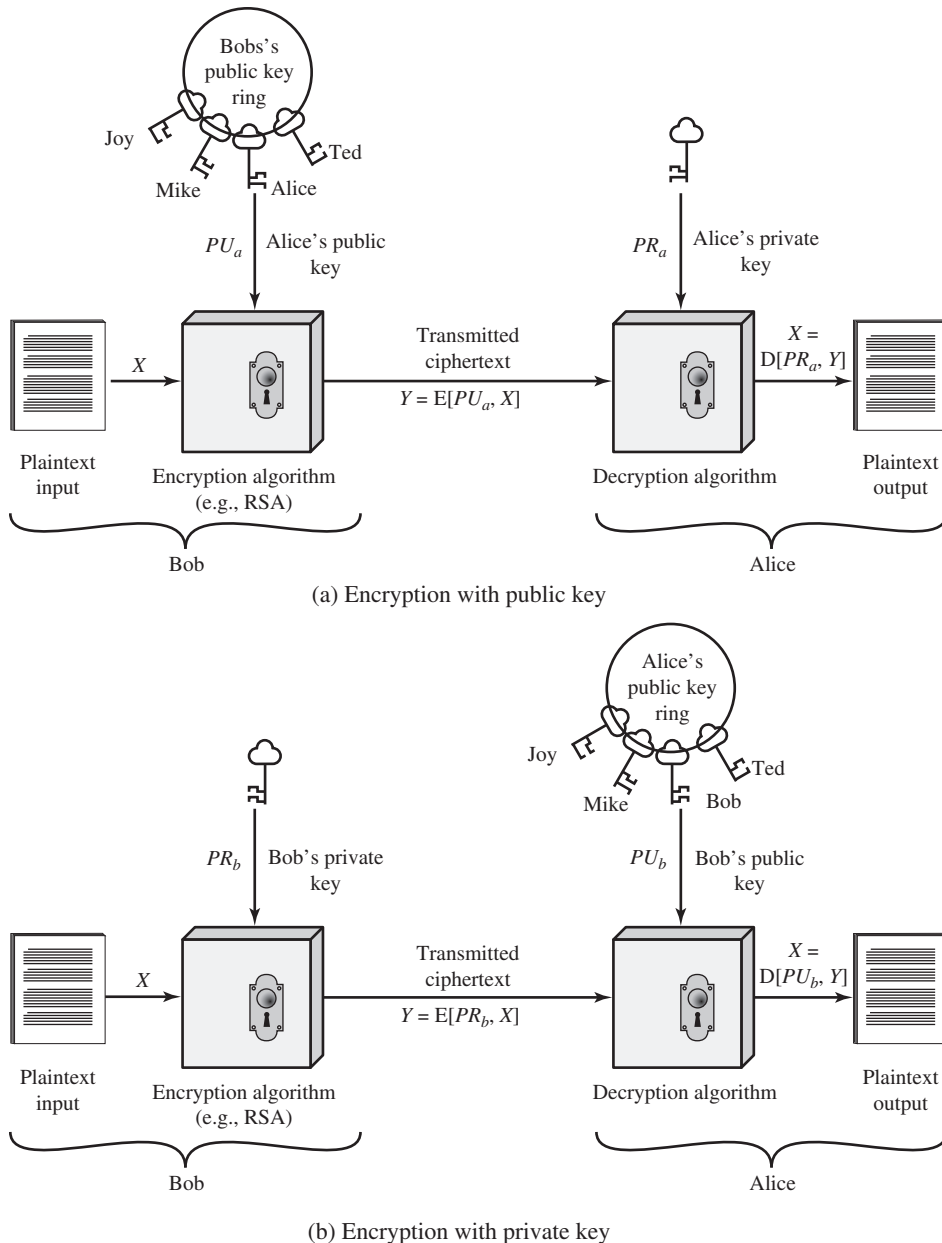


Figure 2.6 Public-Key Cryptography

plaintext, thus providing authentication. Further, no one but Bob would be able to modify the plaintext because only Bob could encrypt the plaintext with Bob's private key. Once again, the actual provision of authentication or data integrity depends on a variety of factors. This issue will be addressed primarily in Chapter 21, but other references are made to it where appropriate in this text.

Applications for Public-Key Cryptosystems

Public-key systems are characterized by the use of a cryptographic type of algorithm with two keys, one held private and one available publicly. Depending on the application, the sender uses either the sender's private key or the receiver's public key, or both, to perform some type of cryptographic function. In broad terms, we can classify the use of public-key cryptosystems into three categories: digital signature, symmetric key distribution, and encryption of secret keys.

These applications will be discussed in Section 2.4. Some algorithms are suitable for all three applications, whereas others can be used only for one or two of these applications. Table 2.3 indicates the applications supported by the algorithms discussed in this section.

Requirements for Public-Key Cryptography

The cryptosystem illustrated in Figure 2.6 depends on a cryptographic algorithm based on two related keys. Diffie and Hellman postulated this system without demonstrating that such algorithms exist. However, they did lay out the conditions that such algorithms must fulfill [DIFF76]:

1. It is computationally easy for a party B to generate a pair (public key PU_b , private key PR_b).
2. It is computationally easy for a sender A, knowing the public key and the message to be encrypted, M , to generate the corresponding ciphertext:

$$C = E(PU_b, M)$$

3. It is computationally easy for the receiver B to decrypt the resulting ciphertext using the private key to recover the original message:

$$M = D(PR_b, C) = D[PR_b, E(PU_b, M)]$$

4. It is computationally infeasible for an opponent, knowing the public key, PU_b , to determine the private key, PR_b .
5. It is computationally infeasible for an opponent, knowing the public key, PU_b , and a ciphertext, C , to recover the original message, M .

Table 2.3 Applications for Public-Key Cryptosystems

Algorithm	Digital Signature	Symmetric Key Distribution	Encryption of Secret Keys
RSA	Yes	Yes	Yes
Diffie-Hellman	No	Yes	No
DSS	Yes	No	No
Elliptic Curve	Yes	Yes	Yes

We can add a sixth requirement that, although useful, is not necessary for all public-key applications:

6. Either of the two related keys can be used for encryption, with the other used for decryption.

$$M = D[PU_b, E(PR_b, M)] = D[PR_b, E(PU_b, M)]$$

Asymmetric Encryption Algorithms

In this subsection, we briefly mention the most widely used asymmetric encryption algorithms. Chapter 21 will provide technical details.

RSA One of the first public-key schemes was developed in 1977 by Ron Rivest, Adi Shamir, and Len Adleman at MIT and first published in 1978 [RIVE78]. The RSA scheme has since reigned supreme as the most widely accepted and implemented approach to public-key encryption. RSA is a block cipher in which the plaintext and ciphertext are integers between 0 and $n - 1$ for some n .

In 1977, the three inventors of RSA dared *Scientific American* readers to decode a cipher they printed in Martin Gardner’s “Mathematical Games” column. They offered a \$100 reward for the return of a plaintext sentence, an event they predicted might not occur for some 40 quadrillion years. In April of 1994, a group working over the Internet and using over 1600 computers claimed the prize after only eight months of work [LEUT94]. This challenge used a public-key size (length of n) of 129 decimal digits, or around 428 bits. This result does not invalidate the use of RSA; it simply means that larger key sizes must be used. Currently, a 1024-bit key size (about 300 decimal digits) is considered strong enough for virtually all applications.

DIFFIE–HELLMAN KEY AGREEMENT The first published public-key algorithm appeared in the seminal paper by Diffie and Hellman that defined public-key cryptography [DIFF76] and is generally referred to as Diffie–Hellman key exchange, or key agreement. A number of commercial products employ this key exchange technique.

The purpose of the algorithm is to enable two users to securely reach agreement about a shared secret that can be used as a secret key for subsequent symmetric encryption of messages. The algorithm itself is limited to the exchange of the keys.

DIGITAL SIGNATURE STANDARD The National Institute of Standards and Technology (NIST) published this originally as FIPS PUB 186 (*Digital Signature Standard (DSS)*, May 1994). The DSS makes use of SHA-1 and presents a new digital signature technique, the Digital Signature Algorithm (DSA). The DSS was originally proposed in 1991 and revised in 1993 in response to public feedback concerning the security of the scheme. There were further revisions in 1998, 2000, 2009, and most recently in 2013 as FIPS PUB 186–4. The DSS uses an algorithm that is designed to provide only the digital signature function. Unlike RSA, it cannot be used for encryption or key exchange.

ELLIPTIC CURVE CRYPTOGRAPHY The vast majority of the products and standards that use public-key cryptography for encryption and digital signatures use RSA. The bit length for secure RSA use has increased over recent years, and this has put a heavier processing load on applications using RSA. This burden has ramifications, especially for electronic commerce sites that conduct large numbers of secure

transactions. Recently, a competing system has begun to challenge RSA: elliptic curve cryptography (ECC). Already, ECC is showing up in standardization efforts, including the IEEE (Institute of Electrical and Electronics Engineers) P1363 Standard for Public-Key Cryptography.

The principal attraction of ECC compared to RSA is that it appears to offer equal security for a far smaller bit size, thereby reducing processing overhead. On the other hand, although the theory of ECC has been around for some time, it is only recently that products have begun to appear and that there has been sustained cryptanalytic interest in probing for weaknesses. Thus, the confidence level in ECC is not yet as high as that in RSA.

2.4 DIGITAL SIGNATURES AND KEY MANAGEMENT

As mentioned in Section 2.3, public-key algorithms are used in a variety of applications. In broad terms, these applications fall into two categories: digital signatures, and various techniques to do with key management and distribution.

With respect to key management and distribution, there are at least three distinct aspects to the use of public-key encryption in this regard:

- The secure distribution of public keys
- The use of public-key encryption to distribute secret keys
- The use of public-key encryption to create temporary keys for message encryption

This section provides a brief overview of digital signatures and the various types of key management and distribution.

Digital Signature

Public-key encryption can be used for authentication with a technique known as the digital signature. NIST FIPS PUB 186-4 [*Digital Signature Standard (DSS)*, July 2013] defines a digital signature as follows: The result of a cryptographic transformation of data that, when properly implemented, provides a mechanism for verifying origin authentication, data integrity and signatory non-repudiation.

Thus, a digital signature is a data-dependent bit pattern, generated by an agent as a function of a file, message, or other form of data block. Another agent can access the data block and its associated signature and verify (1) the data block has been signed by the alleged signer, and (2) the data block has not been altered since the signing. Further, the signer cannot repudiate the signature.

FIPS 186-4 specifies the use of one of three digital signature algorithms:

- **Digital Signature Algorithm (DSA):** The original NIST-approved algorithm, which is based on the difficulty of computing discrete logarithms.
- **RSA Digital Signature Algorithm:** Based on the RSA public-key algorithm.
- **Elliptic Curve Digital Signature Algorithm (ECDSA):** Based on elliptic-curve cryptography.

Figure 2.7 is a generic model of the process of making and using digital signatures. All of the digital signature schemes in FIPS 186-4 have this structure. Suppose

Bob wants to send a message to Alice. Although it is not important that the message be kept secret, he wants Alice to be certain that the message is indeed from him. For this purpose, Bob uses a secure hash function, such as SHA-512, to generate a hash value for the message. That hash value, together with Bob's private key, serve as input to a digital signature generation algorithm that produces a short block that functions as a digital signature. Bob sends the message with the signature attached. When Alice receives the message plus signature, she (1) calculates a hash value for the message; (2) provides the hash value and Bob's public key as inputs to a digital signature verification algorithm. If the algorithm returns the result that the signature is valid, Alice is assured that the message must have been signed by Bob. No one else

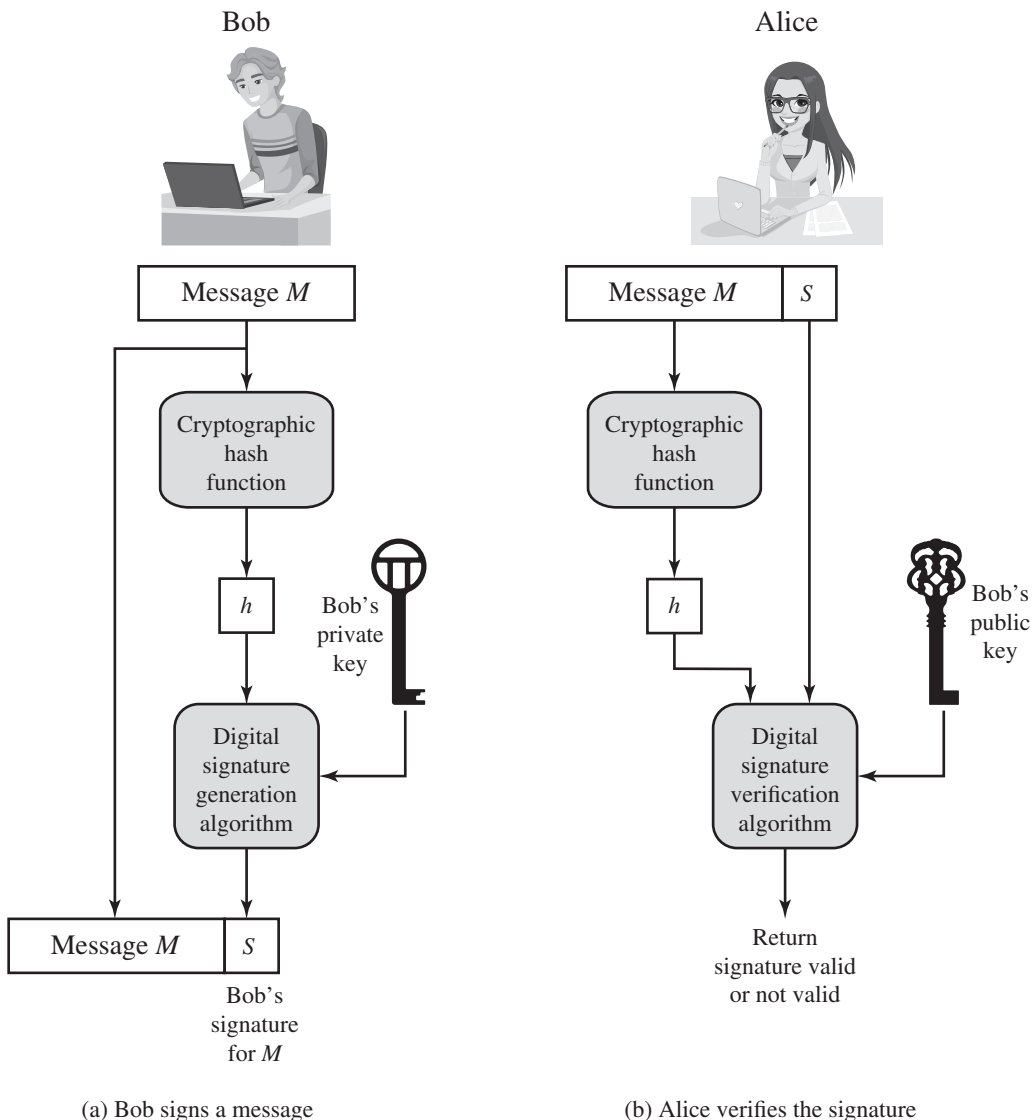


Figure 2.7 Simplified Depiction of Essential Elements of Digital Signature Process

has Bob's private key, and therefore no one else could have created a signature that could be verified for this message with Bob's public key. In addition, it is impossible to alter the message without access to Bob's private key, so the message is authenticated both in terms of source and in terms of data integrity.

The digital signature does not provide confidentiality. That is, the message being sent is safe from alteration, but not safe from eavesdropping. This is obvious in the case of a signature based on a portion of the message, because the rest of the message is transmitted in the clear. Even in the case of complete encryption, there is no protection of confidentiality because any observer can decrypt the message by using the sender's public key.

Public-Key Certificates

On the face of it, the point of public-key encryption is that the public key is public. Thus, if there is some broadly accepted public-key algorithm, such as RSA, any participant can send his or her public key to any other participant or broadcast the key to the community at large. Although this approach is convenient, it has a major weakness. Anyone can forge such a public announcement. That is, some user could pretend to be Bob and send a public key to another participant or broadcast such a public key. Until such time as Bob discovers the forgery and alerts other participants, the forger is able to read all encrypted messages intended for Bob and can use the forged keys for authentication.

The solution to this problem is the public-key certificate. In essence, a certificate consists of a public key plus a user ID of the key owner, with the whole block signed by a trusted third party. The certificate also includes some information about the third party plus an indication of the period of validity of the certificate. Typically, the third party is a certificate authority (CA) that is trusted by the user community, such as a government agency or a financial institution. A user can present his or her public key to the authority in a secure manner and obtain a signed certificate. The user can then publish the certificate. Anyone needing this user's public key can obtain the certificate and verify that it is valid by means of the attached trusted signature. Figure 2.8 illustrates the process.

The key steps can be summarized as follows:

1. User software (client) creates a pair of keys: one public and one private.
2. Client prepares an unsigned certificate that includes the user ID and user's public key.
3. User provides the unsigned certificate to a CA in some secure manner. This might require a face-to-face meeting, the use of registered e-mail, or happen via a Web form with e-mail verification.
4. CA creates a signature as follows:
 - a. CA uses a hash function to calculate the hash code of the unsigned certificate. A hash function is one that maps a variable-length data block or message into a fixed-length value called a hash code, such as SHA family that we will discuss in Sections 2.2 and 21.1.
 - b. CA generates digital signature using the CA's private key and a signature generation algorithm.
5. CA attaches the signature to the unsigned certificate to create a signed certificate.

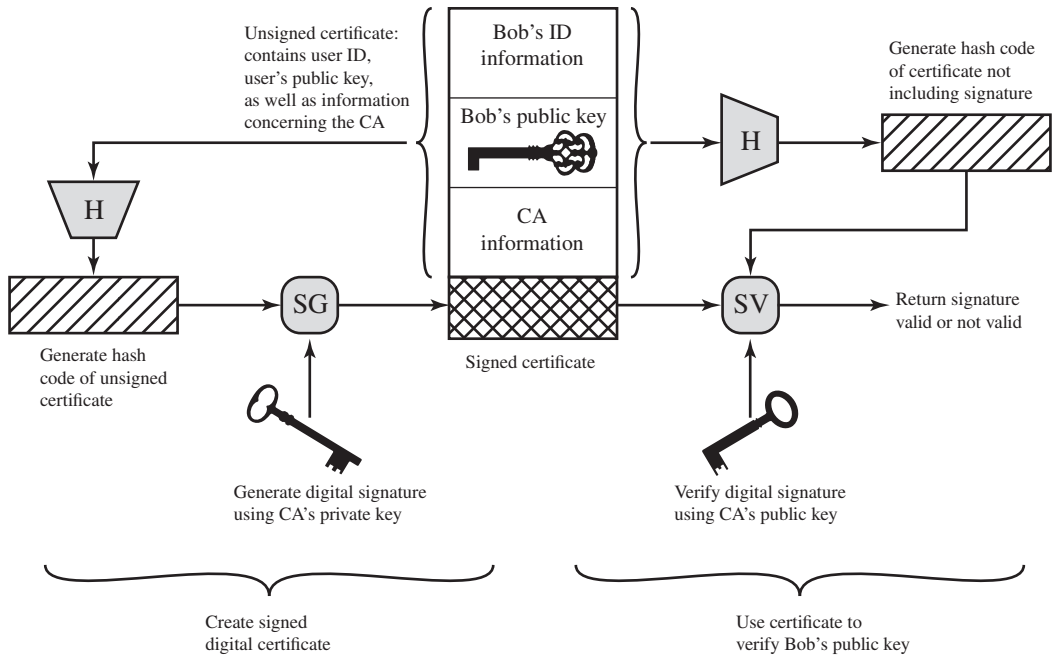


Figure 2.8 Public-Key Certificate Use guarda slide 125-134

6. CA returns the signed certificate to client.
7. Client may provide the signed certificate to any other user.
8. Any user may verify that the certificate is valid as follows:
 - a. User calculates the hash code of certificate (not including signature).
 - b. User verifies digital signature using CA's public key and the signature verification algorithm. The algorithm returns a result of either signature valid or invalid.

One scheme has become universally accepted for formatting public-key certificates: the X.509 standard. X.509 certificates are used in most network security applications, including IP Security (IPsec), Transport Layer Security (TLS), Secure Shell (SSH), and Secure/Multipurpose Internet Mail Extension (S/MIME). We will examine most of these applications in Part Five.

Symmetric Key Exchange Using Public-Key Encryption

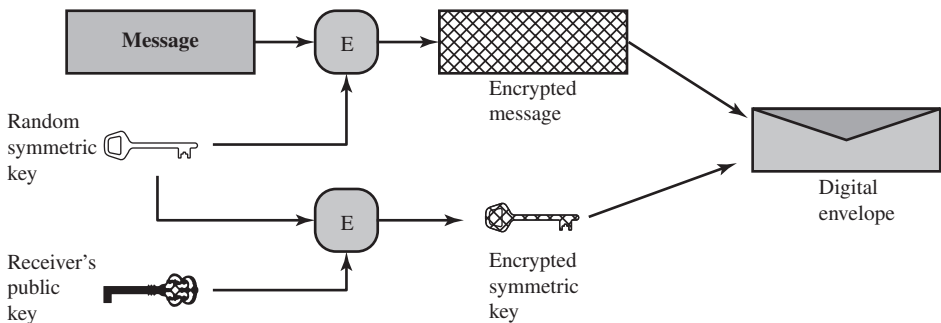
With symmetric encryption, a fundamental requirement for two parties to communicate securely is that they share a secret key. Suppose Bob wants to create a messaging application that will enable him to exchange e-mail securely with anyone who has access to the Internet, or to some other network that the two of them share. Suppose Bob wants to do this using symmetric encryption. With symmetric encryption, Bob and his correspondent, say, Alice, must come up with a way to share a unique secret key that no one else knows. How are they going to do that? If Alice is in the next room from Bob, Bob could generate a key and write it down on a piece of paper or

store it on a disk or thumb drive and hand it to Alice. But if Alice is on the other side of the continent or the world, what can Bob do? He could encrypt this key using symmetric encryption and e-mail it to Alice, but this means that Bob and Alice must share a secret key to encrypt this new secret key. Furthermore, Bob and everyone else who uses this new e-mail package faces the same problem with every potential correspondent: Each pair of correspondents must share a unique secret key.

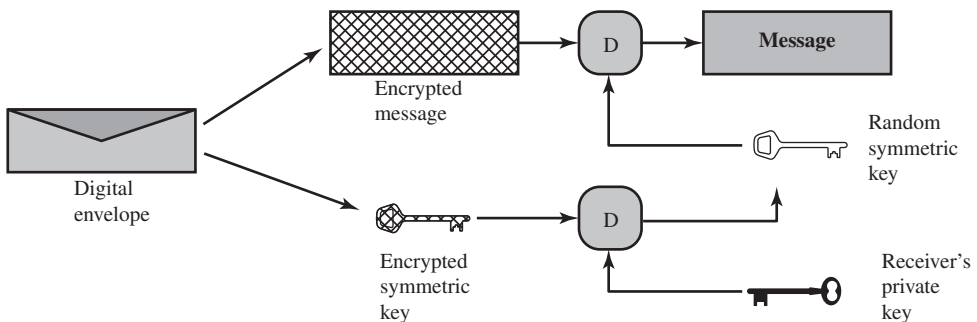
One approach is the use of Diffie–Hellman key exchange. This approach is indeed widely used. However, it suffers the drawback that, in its simplest form, Diffie–Hellman provides no authentication of the two communicating partners. There are variations to Diffie–Hellman that overcome this problem. In addition, there are protocols using other public-key algorithms that achieve the same objective.

Digital Envelopes

Another application in which public-key encryption is used to protect a symmetric key is the digital envelope, which can be used to protect a message without needing to first arrange for sender and receiver to have the same secret key. The technique is referred to as a digital envelope, which is the equivalent of a sealed envelope containing an unsigned letter. The general approach is shown in Figure 2.9. Suppose Bob



(a) Creation of a digital envelope



(b) Opening a digital envelope

Figure 2.9 Digital Envelopes

wishes to send a confidential message to Alice, but they do not share a symmetric secret key. Bob does the following:

1. Prepare a message.
2. Generate a random symmetric key that will be used this one time only.
3. Encrypt that message using symmetric encryption the one-time key.
4. Encrypt the one-time key using public-key encryption with Alice's public key.
5. Attach the encrypted one-time key to the encrypted message and send it to Alice.

Only Alice is capable of decrypting the one-time key and therefore of recovering the original message. If Bob obtained Alice's public key by means of Alice's public-key certificate, then Bob is assured that it is a valid key.

2.5 RANDOM AND PSEUDORANDOM NUMBERS

Random numbers play an important role in the use of encryption for various network security applications. We provide a brief overview in this section. The topic is examined in detail in Appendix D.

The Use of Random Numbers

A number of network security algorithms based on cryptography make use of random numbers. For example:

- Generation of keys for the RSA public-key encryption algorithm (to be described in Chapter 21) and other public-key algorithms.
- Generation of a stream key for symmetric stream cipher.
- Generation of a symmetric key for use as a temporary session key or in creating a digital envelope.
- In a number of key distribution scenarios, such as Kerberos (to be described in Chapter 23), random numbers are used for handshaking to prevent replay attacks.
- Session key generation, whether done by a key distribution center or by one of the principals.

These applications give rise to two distinct and not necessarily compatible requirements for a sequence of random numbers: randomness, and unpredictability.

RANDOMNESS Traditionally, the concern in the generation of a sequence of allegedly random numbers has been that the sequence of numbers be random in some well-defined statistical sense. The following two criteria are used to validate that a sequence of numbers is random:

- **Uniform distribution:** The distribution of numbers in the sequence should be uniform; that is, the frequency of occurrence of each of the numbers should be approximately the same.
- **Independence:** No one value in the sequence can be inferred from the others.

Although there are well-defined tests for determining that a sequence of numbers matches a particular distribution, such as the uniform distribution, there is no such test to “prove” independence. Rather, a number of tests can be applied to demonstrate if a sequence does not exhibit independence. The general strategy is to apply a number of such tests until the confidence that independence exists is sufficiently strong.

In the context of our discussion, the use of a sequence of numbers that appear statistically random often occurs in the design of algorithms related to cryptography. For example, a fundamental requirement of the RSA public-key encryption scheme is the ability to generate prime numbers. In general, it is difficult to determine if a given large number N is prime. A brute-force approach would be to divide N by every odd integer less than $\sqrt{N}$. If N is on the order, say, of 10^{150} , a not uncommon occurrence in public-key cryptography, such a brute-force approach, is beyond the reach of human analysts and their computers. However, a number of effective algorithms exist that test the primality of a number by using a sequence of randomly chosen integers as input to relatively simple computations. If the sequence is sufficiently long (but far, far less than $\sqrt{10^{150}}$), the primality of a number can be determined with near certainty. This type of approach, known as randomization, crops up frequently in the design of algorithms. In essence, if a problem is too hard or time-consuming to solve exactly, a simpler, shorter approach based on randomization is used to provide an answer with any desired level of confidence.

UNPREDICTABILITY In applications such as reciprocal authentication and session key generation, the requirement is not so much that the sequence of numbers be statistically random, but that the successive members of the sequence are unpredictable. With “true” random sequences, each number is statistically independent of other numbers in the sequence and therefore unpredictable. However, as discussed shortly, true random numbers are not always used; rather, sequences of numbers that appear to be random are generated by some algorithm. In this latter case, care must be taken that an opponent is not be able to predict future elements of the sequence on the basis of earlier elements.

Random versus Pseudorandom

Cryptographic applications typically make use of algorithmic techniques for random number generation. These algorithms are deterministic and therefore produce sequences of numbers that are not statistically random. However, if the algorithm is good, the resulting sequences will pass many reasonable tests of randomness. Such numbers are referred to as **pseudorandom numbers**.

You may be somewhat uneasy about the concept of using numbers generated by a deterministic algorithm as if they were random numbers. Despite what might be called philosophical objections to such a practice, it generally works. That is, under most circumstances, pseudorandom numbers will perform as well as if they were random for a given use. The phrase “as well as” is unfortunately subjective, but the use of pseudorandom numbers is widely accepted. The same principle applies in statistical applications, in which a statistician takes a sample of a population and assumes the results will be approximately the same as if the whole population were measured.

A true random number generator (TRNG) uses a nondeterministic source to produce randomness. Most operate by measuring unpredictable natural processes, such as pulse detectors of ionizing radiation events, gas discharge tubes, and leaky capacitors. Intel has developed a commercially available chip that samples thermal noise by amplifying the voltage measured across undriven resistors [JUN99]. LavaRnd is an open source project for creating truly random numbers using inexpensive cameras, open source code, and inexpensive hardware. The system uses a saturated charge-coupled device (CCD) in a light-tight can as a chaotic source to produce the seed. Software processes the result into truly random numbers in a variety of formats. The first commercially available TRNG that achieves bit production rates comparable with that of PRNGs is the Intel digital random number generator (DRNG) [TAYL11], offered on new multicore chips since May 2012.

2.7 KEY TERMS, REVIEW QUESTIONS, AND PROBLEMS

Key Terms

Advanced Encryption Standard (AES) asymmetric encryption authentication brute-force attack ciphertext	collision resistant confidentiality cryptanalysis Data Encryption Standard (DES) data integrity	Decryption Diffie–Hellman key exchange digital signature Digital Signature Standard (DSS) elliptic curve cryptography
--	--	--

encryption hash function keystream message authentication message authentication code (MAC) modes of operation one-way hash function plaintext	preimage resistant private key pseudorandom number public key public-key certificate public-key encryption random number RSA	second preimage resistant secret key secure hash algorithm (SHA) secure hash function strong collision resistant symmetric encryption triple DES weak collision resistant
--	---	--

Review Questions

- 2.1 How is cryptanalysis different from brute-force attack?
- 2.2 List and briefly explain the different approaches to attacking a symmetric encryption scheme.
- 2.3 What are the two principal requirements for the secure use of symmetric encryption?
- 2.4 List the two important aspects of data authentication.
- 2.5 What is one-way hash function?
- 2.6 Briefly describe the three schemes illustrated in Figure 2.3.
- 2.7 What properties must a hash function have to be useful for message authentication?
- 2.8 What are the principal ingredients of a public-key cryptosystem?
- 2.9 List and briefly define three uses of a public-key cryptosystem.
- 2.10 What advantage might elliptic curve cryptography (ECC) have over RSA?
- 2.11 Do digital signatures provide confidentiality?
- 2.12 What is a public-key certificate?
- 2.13 What are three different ways in which random numbers are used in cryptography?

Problems

- 2.1 Typically, in practice, the length of the message is greater than the block size of the encryption algorithm. The simplest approach to handle such encryption is known as electronic codebook (ECB) mode. Explain this mode. Mention a scenario where it cannot be applied. Explain briefly why it is not a secure mode of encryption.
- 2.2 This problem uses a real-world example of a symmetric cipher, from an old U.S. Special Forces manual (public domain). The document, filename *Special Forces.pdf*, is available at box.com/CompSec4e.
 - a. Using the two keys (memory words) *cryptographic* and *network security*, encrypt the following message:

Be at the third pillar from the left outside the lyceum theatre tonight at seven. If you are distrustful bring two friends.

Make reasonable assumptions about how to treat redundant letters and excess letters in the memory words and how to treat spaces and punctuation. Indicate what your assumptions are.

Note: The message is from the Sherlock Holmes novel *The Sign of Four*.
 - b. Decrypt the ciphertext. Show your work.
 - c. Comment on when it would be appropriate to use this technique and what its advantages are.

- 2.3 Consider a very simple symmetric block encryption algorithm, in which 64-bits blocks of plaintext are encrypted using a 128-bit key. Encryption is defined as

$$C = (P \oplus K_0) \boxplus K_1$$

where C = ciphertext; K = secret key; K_0 = leftmost 64 bits of K ; K_1 = rightmost 64 bits of K ; $\oplus$ = bitwise exclusive or; and $\boxplus$ is addition mod 2^{64} .

- a. Show the decryption equation. That is, show the equation for P as a function of C , K_1 and K_2 .
- b. Suppose an adversary has access to two sets of plaintexts and their corresponding ciphertexts and wishes to determine K . We have the two equations:

$$C = (P \oplus K_0) \boxplus K_1; C' = (P' \oplus K_0) \boxplus K_1$$

First, derive an equation in one unknown (e.g., K_0). Is it possible to proceed further to solve for K_0 ?

- 2.4 Perhaps the simplest “serious” symmetric block encryption algorithm is the Tiny Encryption Algorithm (TEA). TEA operates on 64-bit blocks of plaintext using a 128-bit key. The plaintext is divided into two 32-bit blocks (L_0, R_0), and the key is divided into four 32-bit blocks (K_0, K_1, K_2, K_3). Encryption involves repeated application of a pair of rounds, defined as follows for rounds i and $i + 1$:

$$\begin{aligned} L_i &= R_{i-1} \\ R_i &= L_{i-1} \boxplus F(R_{i-1}, K_0, K_1, \delta_i) \\ L_{i+1} &= R_i \\ R_{i+1} &= L_i \boxplus F(R_i, K_2, K_3, \delta_{i+1}) \end{aligned}$$

where F is defined as

$$F(M, K_j, K_k, \delta_i) = ((M \lll 4) \boxplus K_j) \oplus ((M \ggg 5) \boxplus K_k) \oplus (M + \delta_i)$$

and where the logical shift of x by y bits is denoted by $x \lll y$; the logical right shift x by y bits is denoted by $x \ggg y$; and δ_i is a sequence of predetermined constants.

- a. Comment on the significance and benefit of using the sequence of constants.
 - b. Illustrate the operation of TEA using a block diagram or flow chart type of depiction.
 - c. If only one pair of rounds is used, then the ciphertext consists of the 64-bit block (L_2, R_2). For this case, express the decryption algorithm in terms of equations.
 - d. Repeat part (c) using an illustration similar to that used for part (b).
- 2.5 In this problem, we will compare the security services that are provided by digital signatures (DS) and message authentication codes (MAC). We assume Oscar is able to observe all messages sent from Alice to Bob and vice versa. Oscar has no knowledge of any keys but the public one in case of DS. State whether and how (i) DS and (ii) MAC protect against each attack. The value $\text{auth}(x)$ is computed with a DS or a MAC algorithm, respectively.
- a. (Message integrity) Alice sends a message x = “Transfer \$1000 to Mark” in the clear and also sends $\text{auth}(x)$ to Bob. Oscar intercepts the message and replaces “Mark” with “Oscar.” Will Bob detect this?
 - b. (Replay) Alice sends a message x = “Transfer \$1000 to Oscar” in the clear and also sends $\text{auth}(x)$ to Bob. Oscar observes the message and signature and sends them 100 times to Bob. Will Bob detect this?
 - c. (Sender authentication with cheating third party) Oscar claims that he sent some message x with a valid $\text{auth}(x)$ to Bob but Alice claims the same. Can Bob clear the question in either case?
 - d. (Authentication with Bob cheating) Bob claims that he received a message x with a valid signature $\text{auth}(x)$ from Alice (e.g., “Transfer \$1000 from Alice to Bob”) but Alice claims she has never sent it. Can Alice clear this question in either case?

- 2.6 Suppose $H(M)$ is a cryptographic hash function that maps a message of an arbitrary bit length on to an n -bit hash value. Briefly explain the primary security requirements of the hash function H . Assume that H outputs 16-bit hash values. How many random messages would be required to find two different messages M and M' such that $H(M) = H(M')$.
- 2.7 This problem introduces a hash function similar in spirit to SHA that operates on letters instead of binary data. It is called the *toy tetragraph hash* (tth).⁸ Given a message consisting of a sequence of letters, tth produces a hash value consisting of four letters. First, tth divides the message into blocks of 16 letters, ignoring spaces, punctuation, and capitalization. If the message length is not divisible by 16, it is padded out with nulls. A four-number running total is maintained that starts out with the value (0, 0, 0, 0); this is input to a function, known as a *compression function*, for processing the first block. The compression function consists of two rounds. **Round 1:** Get the next block of text and arrange it as a row-wise 4×4 block of text and convert it to numbers (A = 0, B = 1), for example, for the block ABCDEFGHIJKLMNOP, we have

A	B	C	D
E	F	G	H
I	J	K	L
M	N	O	P

0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

Then, add each column mod 26 and add the result to the running total, mod 26. In this example, the running total is (24, 2, 6, 10). **Round 2:** Using the matrix from round 1, rotate the first row left by 1, second row left by 2, third row left by 3, and reverse the order of the fourth row. In our example,

B	C	D	A
G	H	E	F
L	I	J	K
P	O	N	M

1	2	3	0
6	7	4	5
11	8	9	10
15	14	13	12

Now, add each column mod 26 and add the result to the running total. The new running total is (5, 7, 9, 11). This running total is now the input into the first round of the compression function for the next block of text. After the final block is processed, convert the final running total to letters. For example, if the message is ABCDEFGHIJKLMNOP, then the hash is FHJL.

- Draw figures of the overall tth logic and the compression function logic.
 - Calculate the hash function for the 48-letter message “I leave twenty million dollars to my friendly cousin Bill.”
 - To demonstrate the weakness of tth, find a 48-letter block that produces the same hash as that just derived. *Hint:* Use lots of As.
- 2.8 Prior to the discovery of any specific public-key schemes, such as RSA, an existence proof was developed whose purpose was to demonstrate that public-key encryption is possible in theory. Consider the functions $f_1(x_1) = z_1$; $f_2(x_2, y_2) = z_2$; $f_3(x_3, y_3) = z_3$, where all values are integers with $1 \leq x_i, y_i, z_i \leq N$. Function f_1 can be represented by a vector **M1** of length N , in which the k th entry is the value of $f_1(k)$. Similarly,

⁸I thank William K. Mason and The American Cryptogram Association for providing this example.

f_2 and f_3 can be represented by $N \times N$ matrices **M2** and **M3**. The intent is to represent the encryption/decryption process by table look-ups for tables with very large values of N . Such tables would be impractically huge but could, in principle, be constructed. The scheme works as follows: Construct **M1** with a random permutation of all integers between 1 and N ; that is, each integer appears exactly once in **M1**. Construct **M2** so each row contains a random permutation of the first N integers. Finally, fill in **M3** to satisfy the following condition:

$$f_3(f_2(f_1(k), p), k) = p \text{ for all } k, p \text{ with } 1 \leq k, p \leq N$$

In words,

1. **M1** takes an input k and produces an output x .
2. **M2** takes inputs x and p giving output z .
3. **M3** takes inputs z and k and produces p .

The three tables, once constructed, are made public.

- a. It should be clear that it is possible to construct **M3** to satisfy the preceding condition. As an example, fill in **M3** for the following simple case:

M1 =	5	M2 =	5	2	3	4	1	M3 =	5				
	4		4	2	5	1	3		1				
	2		1	3	2	4	5		3				
	3		3	1	4	2	5		4				
	1		2	5	3	4	1		2				

Convention: The i th element of **M1** corresponds to $k = i$. The i th row of **M2** corresponds to $x = i$; the j th column of **M2** corresponds to $p = j$. The i th row of **M3** corresponds to $z = i$; the j th column of **M3** corresponds to $k = j$. We can look at this in another way. The i th row of **M1** corresponds to the i th column of **M3**. The value of the entry in the i th row selects a row of **M2**. The entries in the selected **M2** column are derived from the entries in the selected **M2** row. The first entry in the **M2** row dictates where the value 1 goes in the **M3** column. The second entry in the **M2** row dictates where the value 2 goes in the **M3** column, and so on.

- b. Describe the use of this set of tables to perform encryption and decryption between two users.
 - c. Argue that this is a secure scheme.
- 2.9 Construct a figure similar to Figure 2.9 that includes a digital signature to authenticate the message in the digital envelope.

USER AUTHENTICATION

3.1 Digital User Authentication Principles

- A Model for Digital User Authentication
- Means of Authentication
- Risk Assessment for User Authentication

3.2 Password-Based Authentication

- The Vulnerability of Passwords
- The Use of Hashed Passwords
- Password Cracking of User-Chosen Passwords
- Password File Access Control
- Password Selection Strategies

3.3 Token-Based Authentication

- Memory Cards
- Smart Cards
- Electronic Identify Cards

3.4 Biometric Authentication

- Physical Characteristics Used in Biometric Applications
- Operation of a Biometric Authentication System
- Biometric Accuracy

3.5 Remote User Authentication

- Password Protocol
- Token Protocol
- Static Biometric Protocol
- Dynamic Biometric Protocol

3.6 Security Issues for User Authentication

3.7 Practical Application: An Iris Biometric System

3.8 Case Study: Security Problems for ATM Systems

3.9 Key Terms, Review Questions, and Problems

LEARNING OBJECTIVES

After studying this chapter, you should be able to:

- ◆ Discuss the four general means of authenticating a user's identity.
- ◆ Explain the mechanism by which hashed passwords are used for user authentication.
- ◆ Understand the use of the Bloom filter in password management.
- ◆ Present an overview of token-based user authentication.
- ◆ Discuss the issues involved and the approaches for remote user authentication.
- ◆ Summarize some of the key security issues for user authentication.

In most computer security contexts, user authentication is the fundamental building block and the primary line of defense. User authentication is the basis for most types of access control and for user accountability. User authentication encompasses two functions. First, the user identifies herself to the system by presenting a credential, such as user ID. Second, the system verifies the user by the exchange of authentication information.

For example, user Alice Toklas could have the user identifier ABTOKLAS. This information needs to be stored on any server or computer system that Alice wishes to use, and could be known to system administrators and other users. A typical item of authentication information associated with this user ID is a password, which is kept secret (known only to Alice and to the system)¹. If no one is able to obtain or guess Alice's password, then the combination of Alice's user ID and password enables administrators to set up Alice's access permissions and audit her activity. Because Alice's ID is not secret, system users can send her e-mail, but because her password is secret, no one can pretend to be Alice.

In essence, identification is the means by which a user provides a claimed identity to the system; user authentication is the means of establishing the validity of the claim. Note user authentication is distinct from message authentication. As defined in Chapter 2, message authentication is a procedure that allows communicating parties to verify that the contents of a received message have not been altered, and that the source is authentic. This chapter is concerned solely with user authentication.

This chapter first provides an overview of different means of user authentication, then examines each in some detail.

3.1 DIGITAL USER AUTHENTICATION PRINCIPLES

NIST SP 800-63-3 (*Digital Authentication Guideline*, October 2016) defines digital user authentication as the process of establishing confidence in user identities that are presented electronically to an information system. Systems can use the

¹Typically, the password is stored in hashed form on the server and this hash code may not be secret, as explained subsequently in this chapter.

authenticated identity to determine if the authenticated individual is authorized to perform particular functions, such as database transactions or access to system resources. In many cases, the authentication and transaction, or other authorized function, take place across an open network such as the Internet. Equally authentication and subsequent authorization can take place locally, such as across a local area network. Table 3.1, from NIST SP 800-171 (*Protecting Controlled Unclassified Information in Nonfederal Information Systems and Organizations*, December 2016), provides a useful list of security requirements for identification and authentication services.

Means of Authentication

There are four general means of authenticating a user's identity, which can be used alone or in combination:

- **Something the individual knows:** Examples include a password, a personal identification number (PIN), or answers to a prearranged set of questions.
- **Something the individual possesses:** Examples include electronic keycards, smart cards, and physical keys. This type of authenticator is referred to as a *token*.

- **Something the individual is (static biometrics):** Examples include recognition by fingerprint, retina, and face.
- **Something the individual does (dynamic biometrics):** Examples include recognition by voice pattern, handwriting characteristics, and typing rhythm.

All of these methods, properly implemented and used, can provide secure user authentication. However, each method has problems. An adversary may be able to guess or steal a password. Similarly, an adversary may be able to forge or steal a token. A user may forget a password or lose a token. Further, there is a significant administrative overhead for managing password and token information on systems and securing such information on systems. With respect to biometric authenticators, there are a variety of problems, including dealing with false positives and false negatives, user acceptance, cost, and convenience. **Multifactor authentication** refers to the use of more than one of the authentication means in the preceding list (see Figure 3.2). The strength of authentication systems is largely determined by the number of factors incorporated by the system. Implementations that use two factors are considered to be stronger than those that use only one factor; systems that incorporate three factors are stronger than systems that only incorporate two of the factors, and so on.

3.2 PASSWORD-BASED AUTHENTICATION

A widely used line of defense against intruders is the password system. Virtually all multiuser systems, network-based servers, Web-based e-commerce sites, and other similar services require that a user provide not only a name or identifier (ID) but also a password. The system compares the password to a previously stored password for that user ID, maintained in a system password file. The password serves to authenticate the ID of the individual logging on to the system. In turn, the ID provides security in the following ways:

- The ID determines whether the user is authorized to gain access to a system. In some systems, only those who already have an ID filed on the system are allowed to gain access.
- The ID determines the privileges accorded to the user. A few users may have administrator or “superuser” status that enables them to read files and perform functions that are especially protected by the operating system. Some systems have guest or anonymous accounts, and users of these accounts have more limited privileges than others.
- The ID is used in what is referred to as discretionary access control. For example, by listing the IDs of the other users, a user may grant permission to them to read files owned by that user.

The Vulnerability of Passwords

In this subsection, we outline the main forms of attack against password-based authentication and briefly outline a countermeasure strategy. The remainder of Section 3.2 goes into more detail on the key countermeasures.

Typically, a system that uses password-based authentication maintains a password file indexed by user ID. One technique that is typically used is to store not the user’s password but a one-way hash function of the password, as described subsequently.

We can identify the following attack strategies and countermeasures:

- **Offline dictionary attack:** Typically, strong access controls are used to protect the system’s password file. However, experience shows that determined hackers can frequently bypass such controls and gain access to the file. The attacker obtains the system password file and compares the password hashes against

hashes of commonly used passwords. If a match is found, the attacker can gain access by that ID/password combination. Countermeasures include controls to prevent unauthorized access to the password file, intrusion detection measures to identify a compromise, and rapid reissuance of passwords should the password file be compromised.

- **Specific account attack:** The attacker targets a specific account and submits password guesses until the correct password is discovered. The standard countermeasure is an account lockout mechanism, which locks out access to the account after a number of failed login attempts. Typical practice is no more than five access attempts.
- **Popular password attack:** A variation of the preceding attack is to use a popular password and try it against a wide range of user IDs. A user's tendency is to choose a password that is easily remembered; this unfortunately makes the password easy to guess. Countermeasures include policies to inhibit the selection by users of common passwords and scanning the IP addresses of authentication requests and client cookies for submission patterns.
- **Password guessing against single user:** The attacker attempts to gain knowledge about the account holder and system password policies and uses that knowledge to guess the password. Countermeasures include training in and enforcement of password policies that make passwords difficult to guess. Such policies address the secrecy, minimum length of the password, character set, prohibition against using well-known user identifiers, and length of time before the password must be changed.
- **Workstation hijacking:** The attacker waits until a logged-in workstation is unattended. The standard countermeasure is automatically logging the workstation out after a period of inactivity. Intrusion detection schemes can be used to detect changes in user behavior.
- **Exploiting user mistakes:** If the system assigns a password, then the user is more likely to write it down because it is difficult to remember. This situation creates the potential for an adversary to read the written password. A user may intentionally share a password, to enable a colleague to share files, for example. Also, attackers are frequently successful in obtaining passwords by using social engineering tactics that trick the user or an account manager into revealing a password. Many computer systems are shipped with preconfigured passwords for system administrators. Unless these preconfigured passwords are changed, they are easily guessed. Countermeasures include user training, intrusion detection, and simpler passwords combined with another authentication mechanism.
- **Exploiting multiple password use:** Attacks can also become much more effective or damaging if different network devices share the same or a similar password for a given user. Countermeasures include a policy that forbids the same or similar password on particular network devices.
- **Electronic monitoring:** If a password is communicated across a network to log on to a remote system, it is vulnerable to eavesdropping. Simple encryption will not fix this problem, because the encrypted password is, in effect, the password and can be observed and reused by an adversary.

Despite the many security vulnerabilities of passwords, they remain the most commonly used user authentication technique, and this is unlikely to change in the foreseeable future [HERL12]. Among the reasons for the persistent popularity of passwords are the following:

1. Techniques that utilize client-side hardware, such as fingerprint scanners and smart card readers, require the implementation of the appropriate user authentication software to exploit this hardware on both the client and server systems. Until there is widespread acceptance on one side, there is reluctance to implement on the other side, so we end up with a who-goes-first stalemate.
2. Physical tokens, such as smart cards, are expensive and/or inconvenient to carry around, especially if multiple tokens are needed.
3. Schemes that rely on a single sign-on to multiple services, using one of the non-password techniques described in this chapter, create a single point of security risk.
4. Automated password managers that relieve users of the burden of knowing and entering passwords have poor support for roaming and synchronization across multiple client platforms, and their usability had not been adequately researched.

Thus, it is worth our while to study the use of passwords for user authentication in some detail.

The Use of Hashed Passwords

A widely used password security technique is the use of hashed passwords and a salt value. This scheme is found on virtually all UNIX variants as well as on a number of other operating systems. The following procedure is employed (see Figure 3.3a). To load a new password into the system, the user selects or is assigned a password. This password is combined with a fixed-length **salt** value [MORR79]. In older implementations, this value is related to the time at which the password is assigned to the user. Newer implementations use a pseudorandom or random number. The password and salt serve as inputs to a hashing algorithm to produce a fixed-length hash code. The hash algorithm is designed to be slow to execute in order to thwart attacks. The hashed password is then stored, together with a plaintext copy of the salt, in the password file for the corresponding user ID. The hashed password method has been shown to be secure against a variety of cryptanalytic attacks [WAGN00].

When a user attempts to log on to a UNIX system, the user provides an ID and a password (see Figure 3.3b). The operating system uses the ID to index into the password file and retrieve the plaintext salt and the encrypted password. The salt and user-supplied password are used as input to the encryption routine. If the result matches the stored value, the password is accepted.

The salt serves three purposes:

- It prevents duplicate passwords from being visible in the password file. Even if two users choose the same password, those passwords will be assigned different salt values. Hence, the hashed passwords of the two users will differ.
- It greatly increases the difficulty of offline dictionary attacks. For a salt of length b bits, the number of possible passwords is increased by a factor of 2^b , increasing the difficulty of guessing a password in a dictionary attack.

each guess and submit it to the hash function once for each salt value in the dictionary file, multiplying the number of guesses that must be checked.

There are two threats to the UNIX password scheme. First, a user can gain access on a machine using a guest account or by some other means then run a password guessing program, called a password cracker, on that machine. The attacker should be able to check many thousands of possible passwords with little resource consumption. In addition, if an opponent is able to obtain a copy of the password file, then a cracker program can be run on another machine at leisure. This enables the opponent to run through millions of possible passwords in a reasonable period.

UNIX IMPLEMENTATIONS Since the original development of UNIX, many implementations have relied on the following password scheme. Each user selects a password of up to eight printable characters in length. This is converted into a 56-bit value (using 7-bit ASCII) that serves as the key input to an encryption routine. The hash routine, known as `crypt(3)`, is based on DES. A 12-bit salt value is used. The modified DES algorithm is executed with a data input consisting of a 64-bit block of zeros. The output of the algorithm then serves as input for a second encryption. This process is repeated for a total of 25 encryptions. The resulting 64-bit output is then translated into an 11-character sequence. The modification of the DES algorithm converts it into a one-way hash function. The `crypt(3)` routine is designed to discourage guessing attacks. Software implementations of DES are slow compared to hardware versions, and the use of 25 iterations multiplies the time required by 25.

This particular implementation is now considered woefully inadequate. For example, [PERR03] reports the results of a dictionary attack using a supercomputer. The attack was able to process over 50 million password guesses in about 80 minutes. Further, the results showed that for about \$10,000, anyone should be able to do the same in a few months using one uniprocessor machine. Despite its known weaknesses, this UNIX scheme is still often required for compatibility with existing account management software or in multivendor environments.

There are other much stronger hash/salt schemes available for UNIX. The recommended hash function for many UNIX systems, including Linux, Solaris, and FreeBSD (a widely used open source UNIX), is based on the MD5 secure hash algorithm (which is similar to, but not as secure as SHA-1). The MD5 `crypt` routine uses a salt of up to 48 bits and effectively has no limitations on password length. It produces a 128-bit hash value. It is also far slower than `crypt(3)`. To achieve the slowdown, MD5 `crypt` uses an inner loop with 1000 iterations.

Probably the most secure version of the UNIX hash/salt scheme was developed for OpenBSD, another widely used open source UNIX. This scheme, reported in [PROV99], uses a hash function based on the Blowfish symmetric block cipher. The hash function, called `Bcrypt`, is quite slow to execute. `Bcrypt` allows passwords of up to 55 characters in length and requires a random salt value of 128 bits, to produce a 192-bit hash value. `Bcrypt` also includes a cost variable; an increase in the cost variable causes a corresponding increase in the time required to perform a `Bcrypt` hash. The cost assigned to a new password is configurable, so administrators can assign a higher cost to privileged users.

Password Cracking of User-Chosen Passwords

TRADITIONAL APPROACHES The traditional approach to password guessing, or password cracking as it is called, is to develop a large dictionary of possible passwords and to try each of these against the password file. This means that each password must be hashed using each available salt value then compared with stored hash values. If no match is found, the cracking program tries variations on all the words in its dictionary of likely passwords. Such variations include backward spelling of words, additional numbers or special characters, or sequence of characters.

An alternative is to trade off space for time by precomputing potential hash values. In this approach the attacker generates a large dictionary of possible passwords. For each password, the attacker generates the hash values associated with each possible salt value. The result is a mammoth table of hash values known as a **rainbow table**. For example, [OECH03] showed that using 1.4 GB of data, he could crack 99.9% of all alphanumeric Windows password hashes in 13.8 seconds. This approach can be countered using a sufficiently large salt value and a sufficiently large hash length. Both the FreeBSD and OpenBSD approaches should be secure from this attack for the foreseeable future.

To counter the use of large salt values and hash lengths, password crackers exploit the fact that some people choose easily guessable passwords. A particular problem is that users, when permitted to choose their own password, tend to choose short ones. [BONN12] summarizes the results of a number of studies over the past few years involving over 40 million hacked passwords, as well as their own analysis of almost 70 million anonymized passwords of Yahoo! users, and found a tendency toward six to eight characters of length and a strong dislike of non-alphanumeric characters in passwords.

The analysis of the 70 million passwords in [BONN12] estimates that passwords provide fewer than 10 bits of security against an online, trawling attack, and only about 20 bits of security against an optimal offline dictionary attack. In other words, an attacker who can manage 10 guesses per account, typically within the realm of rate-limiting mechanisms, will compromise around 1% of accounts, just as they would against random 10-bit strings. Against an optimal attacker performing unrestricted brute force and wanting to break half of all available accounts, passwords appear to be roughly equivalent to 20-bit random strings. It can be seen then that using offline search enables an adversary to break a large number of accounts, even if a significant amount of iterated hashing is used.

Password length is only part of the problem. Many people, when permitted to choose their own password, pick a password that is guessable, such as their own name, their street name, a common dictionary word, and so forth. This makes the job of password cracking straightforward. The cracker simply has to test the password file against lists of likely passwords. Because many people use guessable passwords, such a strategy should succeed on virtually all systems.

One demonstration of the effectiveness of guessing is reported in [KLEI90]. From a variety of sources, the author collected UNIX password files, containing nearly 14,000 encrypted passwords. The result, which the author rightly characterizes

as frightening, was that in all, nearly one-fourth of the passwords were guessed. The following strategy was used:

1. Try the user's name, initials, account name, and other relevant personal information. In all, 130 different permutations for each user were tried.
2. Try words from various dictionaries. The author compiled a dictionary of over 60,000 words, including the online dictionary on the system itself, and various other lists as shown.
3. Try various permutations on the words from step 2. This included making the first letter uppercase or a control character, making the entire word uppercase, reversing the word, changing the letter "o" to the digit "zero," and so on. These permutations added another 1 million words to the list.
4. Try various capitalization permutations on the words from step 2 that were not considered in step 3. This added almost 2 million additional words to the list.

Thus, the test involved nearly 3 million words. Using the fastest processor available, the time to encrypt all these words for all possible salt values was under an hour. Keep in mind that such a thorough search could produce a success rate of about 25%, whereas even a single hit may be enough to gain a wide range of privileges on a system.

Attacks that use a combination of brute-force and dictionary techniques have become common. A notable example of this dual approach is John the Ripper, an open-source password cracker first developed in 1996, and still in use [OPEN13].

MODERN APPROACHES Sadly, this type of vulnerability has not lessened in the past 25 years or so. Users are doing a better job of selecting passwords, and organizations are doing a better job of forcing users to pick stronger passwords, a concept known as a complex password policy, as discussed subsequently. However, password-cracking techniques have improved to keep pace. The improvements are of two kinds. First, the processing capacity available for password cracking has increased dramatically. Now used increasingly for computing, graphics processors allow password-cracking programs to work thousands of times faster than they did just a decade ago on similarly priced PCs that used traditional CPUs alone. A PC running a single AMD Radeon HD7970 GPU, for instance, can try on average an 8.2×10^9 password combinations each second, depending on the algorithm used to scramble them [GOOD12a]. Only a decade ago, such speeds were possible only when using pricey supercomputers.

The second area of improvement in password cracking is in the use of sophisticated algorithms to generate potential passwords. For example, [NARA05] developed a model for password generation using the probabilities of letters in natural language. The researchers used standard Markov modeling techniques from natural language processing to dramatically reduce the size of the password space to be searched.

But the best results have been achieved by studying examples of actual passwords in use. To develop techniques that are more efficient and effective than simple dictionary and brute-force attacks, researchers and hackers have studied the structure of passwords. To do this, analysts need a large pool of real-word passwords to study, which they now have. The first big breakthrough came in late 2009, when an SQL injection attack against online games service RockYou.com exposed 32 million

plaintext passwords used by its members to log in to their accounts [TIMM10]. Since then, numerous sets of leaked password files have become available for analysis.

Using large datasets of leaked passwords as training data, [WEIR09] reports on the development of a probabilistic context-free grammar for password cracking. In this approach, guesses are ordered according to their likelihood, based on the frequency of their character-class structures in the training data, as well as the frequency of their digit and symbol substrings. This approach has been shown to be efficient in password cracking [KELL12, ZHAN10].

[MAZU13] reports on an analysis of the passwords used by over 25,000 students at a research university with a complex password policy. The analysts used the password-cracking approach introduced in [WEIR09]. They used a database consisting of a collection of leaked password files, including the RockYou file. Figure 3.4 summarizes a key result from the paper. The graph shows the percentage of passwords that have been recovered as a function of the number of guesses. As can be seen, over 10% of the passwords are recovered after only 10^{10} guesses. After 10^{13} guesses, almost 40% of the passwords are recovered.

Password File Access Control

One way to thwart a password attack is to deny the opponent access to the password file. If the hashed password portion of the file is accessible only by a privileged user, then the opponent cannot read it without already knowing the password of a privileged user. Often, the hashed passwords are kept in a separate file from the user IDs, referred to as a **shadow password file**. Special attention is paid to making the shadow

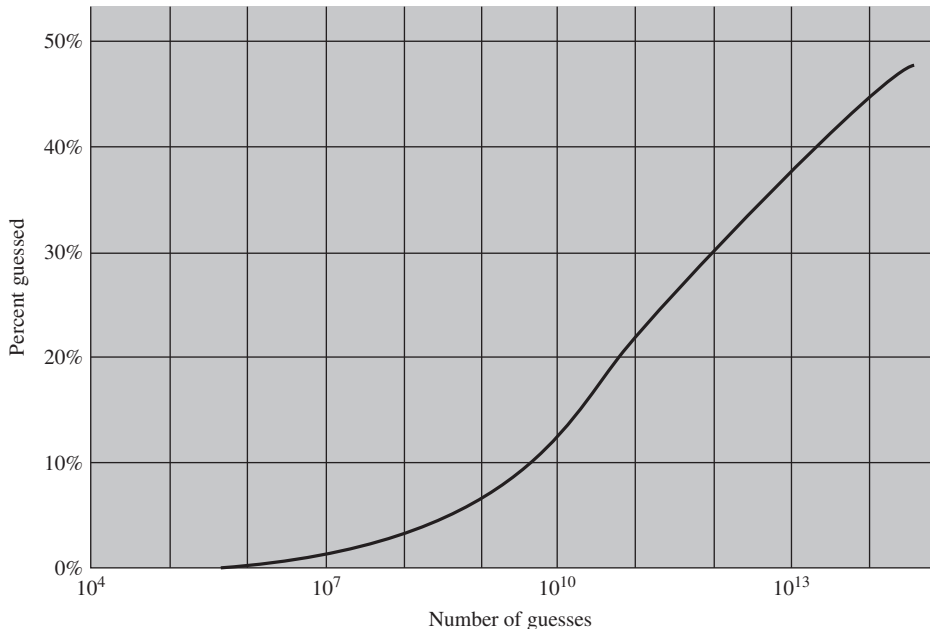


Figure 3.4 The Percentage of Passwords Guessed After a Given Number of Guesses

password file protected from unauthorized access. Although password file protection is certainly worthwhile, there remain vulnerabilities:

- Many systems, including most UNIX systems, are susceptible to unanticipated break-ins. A hacker may be able to exploit a software vulnerability in the operating system to bypass the access control system long enough to extract the password file. Alternatively, the hacker may find a weakness in the file system or database management system that allows access to the file.
- An accident of protection might render the password file readable, thus compromising all the accounts.
- Some of the users have accounts on other machines in other protection domains, and they use the same password. Thus, if the passwords could be read by anyone on one machine, a machine in another location might be compromised.
- A lack of, or weakness in, physical security may provide opportunities for a hacker. Sometimes, there is a backup to the password file on an emergency repair disk or archival disk. Access to this backup enables the attacker to read the password file. Alternatively, a user may boot from a disk running another operating system such as Linux and access the file from this OS.
- Instead of capturing the system password file, another approach to collecting user IDs and passwords is through sniffing network traffic.

Thus, a password protection policy must complement access control measures with techniques to force users to select passwords that are difficult to guess.

Password Selection Strategies

When not constrained, many users choose a password that is too short or too easy to guess. At the other extreme, if users are assigned passwords consisting of eight randomly selected printable characters, password cracking is effectively impossible. But it would be almost as impossible for most users to remember their passwords. Fortunately, even if we limit the password universe to strings of characters that are reasonably memorable, the size of the universe is still too large to permit practical cracking. Our goal, then, is to eliminate guessable passwords while allowing the user to select a password that is memorable. Four basic techniques are in use:

- User education
- Computer-generated passwords
- Reactive password checking
- Complex password policy

Users can be told the importance of using hard-to-guess passwords and can be provided with guidelines for selecting strong passwords. This **user education** strategy is unlikely to succeed at most installations, particularly where there is a large user population or a lot of turnover. Many users will simply ignore the guidelines. Others may not be good judges of what is a strong password. For example, many users (mistakenly) believe that reversing a word or capitalizing the last letter makes a password unguessable.

Nonetheless, it makes sense to provide users with guidelines on the selection of passwords. Perhaps the best approach is the following advice: A good technique for choosing a password is to use the first letter of each word of a phrase. However, do not pick a well-known phrase like “An apple a day keeps the doctor away” (Aaadtlda). Instead, pick something like “My dog’s first name is Rex” (MdfniR) or “My sister Peg is 24 years old” (MsPi24yo). Studies have shown users can generally remember such passwords, but they are not susceptible to password guessing attacks based on commonly used passwords.

Computer-generated passwords also have problems. If the passwords are quite random in nature, users will not be able to remember them. Even if the password is pronounceable, the user may have difficulty remembering it and so be tempted to write it down. In general, computer-generated password schemes have a history of poor acceptance by users. FIPS 181 defines one of the best-designed automated password generators. The standard includes not only a description of the approach but also a complete listing of the C source code of the algorithm. The algorithm generates words by forming pronounceable syllables and concatenating them to form a word. A random number generator produces a random stream of characters used to construct the syllables and words.

A **reactive password checking** strategy is one in which the system periodically runs its own password cracker to find guessable passwords. The system cancels any passwords that are guessed and notifies the user. This tactic has a number of drawbacks. First, it is resource intensive if the job is done right. Because a determined opponent who is able to steal a password file can devote full CPU time to the task for hours or even days, an effective reactive password checker is at a distinct disadvantage. Furthermore, any existing passwords remain vulnerable until the reactive password checker finds them. A good example is the openware Jack the Ripper password cracker (openwall.com/john/pro/), which works on a variety of operating systems.

A promising approach to improved password security is a **complex password policy**, or **proactive password checker**. In this scheme, a user is allowed to select his or her own password. However, at the time of selection, the system checks to see if the password is allowable and, if not, rejects it. Such checkers are based on the philosophy that, with sufficient guidance from the system, users can select memorable passwords from a fairly large password space that are not likely to be guessed in a dictionary attack.

The trick with a proactive password checker is to strike a balance between user acceptability and strength. If the system rejects too many passwords, users will complain that it is too hard to select a password. If the system uses some simple algorithm to define what is acceptable, this provides guidance to password crackers to refine their guessing technique. In the remainder of this subsection, we will look at possible approaches to proactive password checking.

RULE ENFORCEMENT The first approach is a simple system for rule enforcement. For example, NIST SP 800-63-2 suggests the following alternative rules:

- Password must have at least sixteen characters (basic16).
- Password must have at least eight characters including an uppercase and lowercase letter, a symbol, and a digit. It may not contain a dictionary word (comprehensive8).

Although NIST considers basic16 and comprehensive8 equivalent, [KELL12] found that basic16 is superior against large numbers of guesses. Combined with a

prior result that basic16 is also easier for users [KOMA11], this suggests basic16 is the better policy choice.

Although this approach is superior to simply educating users, it may not be sufficient to thwart password crackers. This scheme alerts crackers as to which passwords *not* to try, but may still make it possible to do password cracking.

The process of rule enforcement can be automated by using a proactive password checker, such as the openware pam_passwdqc (openwall.com/passwdqc/), which enforces a variety of rules on passwords and is configurable by the system administrator.

PASSWORD CHECKER Another possible procedure is simply to compile a large dictionary of possible “bad” passwords. When a user selects a password, the system checks to make sure that it is not on the disapproved list. There are two problems with this approach:

- **Space:** The dictionary must be very large to be effective.
- **Time:** The time required to search a large dictionary may itself be large. In addition, to check for likely permutations of dictionary words, either those words must be included in the dictionary, making it truly huge, or each search must also involve considerable processing.

3.3 TOKEN-BASED AUTHENTICATION

Objects that a user possesses for the purpose of user authentication are called tokens. In this section, we examine two types of tokens that are widely used; these are cards that have the appearance and size of bank cards (see Table 3.3).

Memory Cards

Memory cards can store but not process data. The most common such card is the bank card with a magnetic stripe on the back. A magnetic stripe can store only a simple security code, which can be read (and unfortunately reprogrammed) by an inexpensive card reader. There are also memory cards that include an internal electronic memory.

Memory cards can be used alone for physical access, such as a hotel room. For authentication, a user provides both the memory card and some form of password or personal identification number (PIN). A typical application is an automatic teller machine (ATM). The memory card, when combined with a PIN or password, provides significantly greater security than a password alone. An adversary must gain physical possession of the card (or be able to duplicate it) plus must gain knowledge of the PIN. Among the potential drawbacks NIST SP 800-12 (*An Introduction to Computer Security: The NIST Handbook*, October 1995) notes the following:

- **Requires special reader:** This increases the cost of using the token and creates the requirement to maintain the security of the reader's hardware and software.

Table 3.3 Types of Cards Used as Tokens

Card Type	Defining Feature	Example
Embossed	Raised characters only, on front	Old credit card
Magnetic stripe	Magnetic bar on back, characters on front	Bank card
Memory	Electronic memory inside	Prepaid phone card
Smart Contact Contactless	Electronic memory and processor inside Electrical contacts exposed on surface Radio antenna embedded inside	Biometric ID card

²The Bloom filter involves the use of probabilistic techniques. There is a small probability that some passwords not in the dictionary will be rejected. It is often the case in designing algorithms that the use of probabilistic techniques results in a less time-consuming or less complex solution, or both.

- **Token loss:** A lost token temporarily prevents its owner from gaining system access. Thus, there is an administrative cost in replacing the lost token. In addition, if the token is found, stolen, or forged, then an adversary need only determine the PIN to gain unauthorized access.
- **User dissatisfaction:** Although users may have no difficulty in accepting the use of a memory card for ATM access, its use for computer access may be deemed inconvenient.

Smart Cards

A wide variety of devices qualify as smart tokens. These can be categorized along four dimensions that are not mutually exclusive:

- **Physical characteristics:** Smart tokens include an embedded microprocessor. A smart token that looks like a bank card is called a smart card. Other smart tokens can look like calculators, keys, or other small portable objects.
- **User interface:** Manual interfaces include a keypad and display for human/token interaction.
- **Electronic interface:** A smart card or other token requires an electronic interface to communicate with a compatible reader/writer. A card may have one or both of the following types of interface:
 - **Contact:** A contact smart card must be inserted into a smart card reader with a direct connection to a conductive contact plate on the surface of the card (typically gold plated). Transmission of commands, data, and card status takes place over these physical contact points.
 - **Contactless:** A contactless card requires only close proximity to a reader. Both the reader and the card have an antenna, and the two communicate using radio frequencies. Most contactless cards also derive power for the internal chip from this electromagnetic signal. The range is typically one-half to three inches for non-battery-powered cards, ideal for applications such as building entry and payment that require a very fast card interface.
- **Authentication protocol:** The purpose of a smart token is to provide a means for user authentication. We can classify the authentication protocols used with smart tokens into three categories:
 - **Static:** With a static protocol, the user authenticates himself or herself to the token then the token authenticates the user to the computer. The latter half of this protocol is similar to the operation of a memory token.
 - **Dynamic password generator:** In this case, the token generates a unique password periodically (e.g., every minute). This password is then entered into the computer system for authentication, either manually by the user or electronically via the token. The token and the computer system must be initialized and kept synchronized so the computer knows the password that is current for this token.

- **Challenge-response:** In this case, the computer system generates a challenge, such as a random string of numbers. The smart token generates a response based on the challenge. For example, public-key cryptography could be used and the token could encrypt the challenge string with the token's private key.

For user authentication, the most important category of smart token is the smart card, which has the appearance of a credit card, has an electronic interface, and may use any of the type of protocols just described. The remainder of this section discusses smart cards.

A smart card contains within it an entire microprocessor, including processor, memory, and I/O ports. Some versions incorporate a special co-processing circuit for cryptographic operation to speed the task of encoding and decoding messages or generating digital signatures to validate the information transferred. In some cards, the I/O ports are directly accessible by a compatible reader by means of exposed electrical contacts. Other cards rely instead on an embedded antenna for wireless communication with the reader.

A typical smart card includes three types of memory. Read-only memory (ROM) stores data that does not change during the card's life, such as the card number and the cardholder's name. Electrically erasable programmable ROM (EEPROM) holds application data and programs, such as the protocols that the card can execute. It also holds data that may vary with time. For example, in a telephone card, the EEPROM holds the remaining talk time. Random access memory (RAM) holds temporary data generated when applications are executed.

Figure 3.6 illustrates the typical interaction between a smart card and a reader or computer system. Each time the card is inserted into a reader, a reset is initiated by the reader to initialize parameters such as clock value. After the reset function is performed, the card responds with answer to reset (ATR) message. This message defines the parameters and protocols that the card can use and the functions it can perform. The terminal may be able to change the protocol used and other parameters via a protocol type selection (PTS) command. The card's PTS response confirms the protocols and parameters to be used. The terminal and card can now execute the protocol to perform the desired application.

3.4 BIOMETRIC AUTHENTICATION

A biometric authentication system attempts to authenticate an individual based on his or her unique physical characteristics. These include static characteristics, such as fingerprints, hand geometry, facial characteristics, and retinal and iris patterns;

and dynamic characteristics, such as voiceprint and signature. In essence, biometrics is based on pattern recognition. Compared to passwords and tokens, biometric authentication is both technically more complex and expensive. While it is used in a number of specific applications, biometrics has yet to mature as a standard tool for user authentication to computer systems.

Physical Characteristics Used in Biometric Applications

A number of different types of physical characteristics are either in use or under study for user authentication. The most common are the following:

- **Facial characteristics:** Facial characteristics are the most common means of human-to-human identification; thus it is natural to consider them for identification by computer. The most common approach is to define characteristics based on relative location and shape of key facial features, such as eyes, eyebrows, nose, lips, and chin shape. An alternative approach is to use an infrared camera to produce a face thermogram that correlates with the underlying vascular system in the human face.
- **Fingerprints:** Fingerprints have been used as a means of identification for centuries, and the process has been systematized and automated particularly for law enforcement purposes. A fingerprint is the pattern of ridges and furrows on the surface of the fingertip. Fingerprints are believed to be unique across the entire human population. In practice, automated fingerprint recognition and matching system extract a number of features from the fingerprint for storage as a numerical surrogate for the full fingerprint pattern.
- **Hand geometry:** Hand geometry systems identify features of the hand, including shape, and lengths and widths of fingers.
- **Retinal pattern:** The pattern formed by veins beneath the retinal surface is unique and therefore suitable for identification. A retinal biometric system obtains a digital image of the retinal pattern by projecting a low-intensity beam of visual or infrared light into the eye.
- **Iris:** Another unique physical characteristic is the detailed structure of the iris.
- **Signature:** Each individual has a unique style of handwriting and this is reflected especially in the signature, which is typically a frequently written sequence. However, multiple signature samples from a single individual will not be identical. This complicates the task of developing a computer representation of the signature that can be matched to future samples.
- **Voice:** Whereas the signature style of an individual reflects not only the unique physical attributes of the writer but also the writing habit that has developed, voice patterns are more closely tied to the physical and anatomical characteristics of the speaker. Nevertheless, there is still a variation from sample to sample over time from the same speaker, complicating the biometric recognition task.

Figure 3.8 gives a rough indication of the relative cost and accuracy of these biometric measures. The concept of accuracy does not apply to user authentication schemes using smart cards or passwords. For example, if a user enters a password, it either matches exactly the password expected for that user or not. In the case of

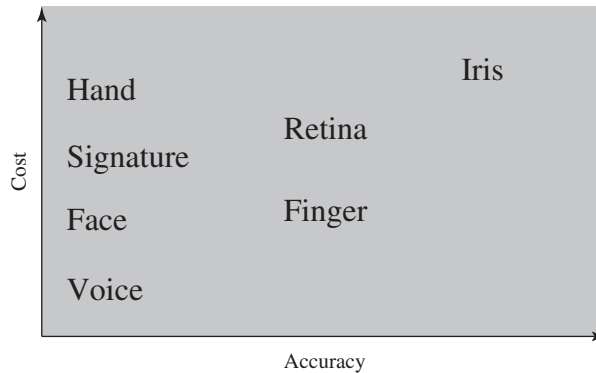


Figure 3.8 Cost Versus Accuracy of Various Biometric Characteristics in User Authentication Schemes

biometric parameters, the system instead must determine how closely a presented biometric characteristic matches a stored characteristic. Before elaborating on the concept of biometric accuracy, we need to have a general idea of how biometric systems work.

Operation of a Biometric Authentication System

Figure 3.9 illustrates the operation of a biometric system. Each individual who is to be included in the database of authorized users must first be **enrolled** in the system. This is analogous to assigning a password to a user. For a biometric system, the user presents a name and, typically, some type of password or PIN to the system. At the same time, the system senses some biometric characteristic of this user (e.g., fingerprint of right index finger). The system digitizes the input then extracts a set of features that can be stored as a number or set of numbers representing this unique biometric characteristic; this set of numbers is referred to as the user's template. The user is now enrolled in the system, which maintains for the user a name (ID), perhaps a PIN or password, and the biometric value.

Depending on application, user authentication on a biometric system involves either **verification** or **identification**. Verification is analogous to a user logging on to a system by using a memory card or smart card coupled with a password or PIN. For biometric verification, the user enters a PIN and also uses a biometric sensor. The system extracts the corresponding feature and compares that to the template stored for this user. If there is a match, then the system authenticates this user.

For an identification system, the individual uses the biometric sensor but presents no additional information. The system then compares the presented template with the set of stored templates. If there is a match, then this user is identified. Otherwise, the user is rejected.